

Lerneinheit 2 · Python-Grundlagen 2: Container und Daten

Version 1.0

Lernziele

Nach diesem Teil können Sie ...

- verschachtelte Daten aus Listen, Tupeln und Dicts lesen, ergänzen und umbauen,
- Texte zerlegen, Vorkommen zählen und mit Sets eindeutige Werte bestimmen,
- Daten mit Slicing, Comprehensions, `zip` und `sorted` auswählen und ordnen und begründen, wann die Schleife die bessere Wahl ist,
- eine Lösung gegen ihre Anforderungen prüfen, mit eigenen Tests und einem begründeten Urteil.

Teil 2 von drei. Im Termin haben Sie Tupel, Dictionaries und Sets an den Kursdaten kennengelernt. Dieser Teil arbeitet diese Themen systematisch auf. Am Ende können Sie mit den Datenstrukturen der Workbooks selbstständig arbeiten: Daten lesen, auswählen, verändern, sortieren und Ihre Ergebnisse prüfen. Vorausgesetzt wird Teil 1 des Grundkurses.

Planen Sie rund drei Stunden ein, am besten in zwei oder drei Sitzungen. Ein guter Schnitt liegt nach dem Kapitel „Text verarbeiten, zählen, Mengen bilden“ und vor dem Abschluss. Alle Aufgaben arbeiten mit den Daten des Kurses. Entwickelt und getestet wird an wenigen Datensätzen, danach wenden Sie Ihre Lösung auf alle 400 Absätze an. Unter jeder Aufgabe steht eine Prüfwelle. Sie meldet, ob Ihre Lösung die vorgegebenen Testfälle besteht.

Dieser Teil ist für das Selbststudium gedacht. Im Notebook steht deshalb unter jeder Aufgabenzelle die Lösung zum Aufklappen, im Skript stehen alle Lösungen im Anhang. Versuchen Sie die Aufgabe erst selbst und sehen Sie danach nach. Viele Aufgaben haben eine Zusatzaufgabe für alle, die das Thema vertiefen wollen. Dafür steht nach der Prüfwelle eine eigene Zelle bereit, ebenfalls mit Lösung.

In den Aufgabenzellen markieren die Zeilen `# TODO: Ihre Lösung hier` und `...` die Stelle für Ihren Code. Die drei Punkte sind in Python ein gültiger Platzhalter mit dem Namen Ellipsis: Eine Zelle mit `...` läuft ohne Fehlermeldung, tut an dieser Stelle aber nichts. Ersetzen Sie beide Zeilen durch Ihre Lösung. Im Text stehen drei Punkte in Klammern, etwa in `len(...)`, für die Argumente, die an dieser Stelle weggelassen sind.

An mehreren Stellen stehen Boxen mit dem Titel „Vertiefung“ oder „Exkurs“. Sie beantworten Fragen für alle, die es genau wissen wollen, und sind nicht prüfungsrelevant. Dateien selbst zu öffnen und Fehler abzufangen ist ein Thema von Teil 3. Hier ist die Zelle, die die Daten lädt, vorgegeben.

Inhalt

- [2.1 Selbsteinstufung](#)
- [2.2 Die Daten und das Ziel](#)
- [2.3 Tupel, Dicts und verschachtelte Daten](#)
- [2.4 Text verarbeiten, zählen, Mengen bilden](#)
- [2.5 Slicing und Kopien](#)
- [2.6 enumerate, zip und Comprehensions](#)
- [2.7 Sortieren und Funktionen als Argumente](#)
- [2.8 Abschluss](#)

2.1 Selbsteinstufung

Acht kleine Schnipsel. Sagen Sie für jeden die Ausgabe vorher, auf Papier, ohne die Zelle auszuführen und ohne KI. Sonst sagt Ihnen der Test nichts. Führen Sie die Zelle danach aus und vergleichen Sie.

```

# Erst JEDE Ausgabe vorhersagen, dann ausführen und vergleichen
pair = ("Gehirn", 0.89)
title, score = pair
print("A:", score, len(pair))

p = {"id": 7, "title": "Zeit"}
print("B:", p["title"], p.get("author", "unbekannt"), "Zeit" in p)

words = "Die GPU, die gpu".lower().split()
print("C:", len(words), len(set(words)))

xs = [10, 20, 30, 40, 50]
print("D:", xs[1:-1], xs[::2])

rows = [{"title": "Zeit"}, {"title": "Koran"}]
first = rows[:1]
first[0]["title"] = "ZEIT"
print("E:", rows[0]["title"])

for name, n in zip(["a", "b", "c"], [1, 2]):
    print("F:", name, n)

print("G:", [x * 2 for x in [1, 2, 3, 4] if x % 2 == 0])

names = ["Tensor", "GPU", "Python"]
print("H:", sorted(names, key=len), names)

```

So lesen Sie Ihr Ergebnis:

Falsch	Dorthin
A oder B	2.3
C	2.4
D oder E	2.5
F oder G	2.6
H	2.7

Alles richtig? Dann lesen Sie das kurze Kapitel [2.2](#) und gehen zum Abschluss in [2.8](#). Er verwendet nichts aus den übersprungenen Kapiteln. Bei einzelnen Unsicherheiten arbeiten Sie die genannten Kapitel durch. Die Vorgabe-Zellen im Kapitel [2.2](#) brauchen Sie in jedem Fall. Drei spätere Stellen greifen außerdem auf Ergebnisse früherer Kapitel zurück: ein Review auf die Funktion `clean_words`, eine Demo auf das Dict `word_counts` und eine Zusatzaufgabe auf das Dict `groups`. Jede dieser Stellen nennt ihre Voraussetzung.

2.2 Die Daten und das Ziel

Die nächsten beiden Zellen stellen die Daten bereit, mit denen dieser Teil arbeitet: die 400 Absätze und die acht Beispielanfragen aus dem Termin. Die Dateien sind im Format JSON gespeichert. Sie enthalten strukturierte Daten, und die Ladezelle macht daraus Python-Dicts und Listen mit Strings und Zahlen. Wie das Öffnen und Lesen von Dateien im Einzelnen funktioniert, behandelt Teil 3. Hier genügt es, die Zelle auszuführen.

```
# Vorgabe: Kursdaten laden – einfach ausführen
import json
① with open("data/appetizer_passages.json", encoding="utf-8") as f:
②     passages = json.load(f)["passages"]           # Liste mit 400 Absätzen
    with open("data/appetizer_queries.json", encoding="utf-8") as f:
        queries = json.load(f)["queries"]           # Liste mit 8 Beispielanfragen

print(len(passages), "Absätze", len(queries), "Beispielanfragen")
```

① `with open(...) as f` öffnet die Datei und schließt sie am Ende des eingerückten Blocks wieder; `encoding="utf-8"` legt die Zeichenkodierung fest, in der die Umlaute gespeichert sind

② `json.load(f)` liest den Inhalt der Datei und baut daraus Dicts und Listen

400 Absätze, 8 Beispielanfragen

Drei Datensätze zeigen, worum es geht. Ein Absatz ist ein Dict mit den Schlüsseln `id`, `title` und `text`. Eine Beispielanfrage ist ein Dict mit dem Fragetext und der Nummer des Absatzes, der gefunden werden soll. Ein Treffer der Suche ist ein Tupel aus Titel und Score, also dem Ähnlichkeitswert.

```
p = passages[108]
print(p["id"], p["title"], p["text"][:60])      # ein Absatz, vom Text nur die ersten 60 Zeichen
print(queries[0])                             # eine Beispielanfrage
hit = ("San Diego", 0.831)                     # ein Treffer: Titel und Score
print(hit)
```

108 San Diego Geographie.

San Diego liegt an einem künstlichen Hafenbecken

```
{'text': 'Wo liegt San Diego?', 'expected': 108}
```

```
('San Diego', 0.831)
```

Das Ziel dieses Teils

Am Ende schreiben Sie eine Funktion, die aus einer Trefferliste die besten Treffer auswählt. Sie hat vier Anforderungen: Sie übernimmt nur Treffer ab einem Mindest-Score, ordnet sie absteigend nach dem Score, gibt höchstens `k` Treffer zurück und lässt die Eingabeliste unverändert. Die Leitfrage dieses Teils lautet: Wie schreiben Sie eine solche Auswahl, und wie zeigen Sie, dass sie jede der vier Anforderungen erfüllt? Die Werkzeuge zum Schreiben sammeln Sie in den folgenden Kapiteln. Zum Zeigen gehören eigene Tests für ausgewählte Fälle und eine Begründung am Code.

Zum Entwickeln sind 400 Absätze zu viele: Ein Ergebnis mit 400 Einträgen lässt sich nicht von Hand kontrollieren. Die nächste Zelle legt deshalb eine kleine Auswahl an. Sie enthält drei Absätze, von denen zwei aus demselben Artikel stammen. An ihr entwickeln und testen Sie Ihre Lösungen, bevor Sie sie auf alle Daten anwenden.

```
# Vorgabe: drei Absätze zum Entwickeln und Testen – einfach ausführen
sample = [passages[6], passages[108], passages[78]]
for p in sample:
    print(p["id"], p["title"])
```

6 Mexiko-Stadt

108 San Diego

78 Mexiko-Stadt

2.3 Tupel, Dicts und verschachtelte Daten

Tupel. Ein Tupel fasst mehrere Werte zu einem festen Paket zusammen. Im Termin war das der Treffer aus Titel und Score. Die nächste Zelle zeigt die Schreibweisen, die Sie im Umgang mit Tupeln brauchen.

```

hit = ("Gehirn", 0.89)                # ein Treffer: Titel und Score
① title, score = hit
   print(title, score)
② a, b = 1, 2
   a, b = b, a
   print(a, b)
③ single = ("Gehirn",)
   no_tuple = ("Gehirn")
   print(type(single), type(no_tuple))

```

- ① das Auspacken verteilt die Werte eines Tupels auf Namen, links stehen so viele Namen wie das Tupel Werte hat
- ② auf der rechten Seite entsteht erst das Tupel `(b, a)`, dann wird es ausgepackt: So tauschen zwei Namen ihre Werte ohne Hilfsvariable
- ③ ein Tupel mit einem Element braucht ein Komma, Klammern allein bilden kein Tupel

```

Gehirn 0.89
2 1
<class 'tuple'> <class 'str'>

```

Tupel oder Liste? Beide speichern Werte in fester Reihenfolge. Sie werden üblicherweise für verschiedene Zwecke verwendet. Ein Tupel ist ein Datensatz: Jede Position hat eine feste Bedeutung, im Treffer steht vorn der Titel und hinten der Score. Eine Liste enthält gleichartige Werte, deren Anzahl sich ändern kann, zum Beispiel alle Treffer einer Suche. Daraus ergibt sich die Faustregel: Wenn die Position die Bedeutung trägt, passt das Tupel. Wenn Werte hinzukommen oder wegfallen können, passt die Liste. Auch eine Funktion, die mehrere Werte zurückgibt, liefert ein Tupel. Das haben Sie in Teil 1 bei `score_stats` genutzt.

Aufgabe 2.3 #1

2 min

Die Trefferliste in der Zelle darunter enthält je Treffer drei Werte: Titel, Score und Absatznummer. Die Schleife soll für jeden Treffer eine Zeile wie `0.89 Gehirn` in der Liste `lines` sammeln. Sie bricht stattdessen mit einem `ValueError` ab. Führen Sie die Zelle aus, lesen Sie die letzte Zeile der Fehlermeldung und korrigieren Sie den Fehler direkt in der Zelle.

Hinweis: Die Meldung nennt die erwartete Anzahl der Werte. Zählen Sie die Namen links vom `in` und vergleichen Sie sie mit der Anzahl der Werte in einem Tupel. Beide müssen übereinstimmen, auch wenn Sie einen der Werte nicht brauchen.

[→ Lösung im Anhang](#)

```

hits = [("Gehirn", 0.89, 92), ("Zeit", 0.72, 157)]    # Titel, Score, Absatznummer
lines = []
for title, score in hits:
    lines.append(f"{score:.2f} {title}")
print(lines)

```

```

# Prü fzelle – einfach ausführen
assert lines == ["0.89 Gehirn", "0.72 Zeit"], "je Treffer eine Zeile aus Score und Titel"
print("Alle Prüfungen bestanden ✓")

```

Dictionaries. Ein Dict ordnet Schlüssel Werten zu. Im Termin haben Sie Werte über den Schlüssel gelesen. Die nächste Zelle zeigt, wie ein Dict verändert und durchlaufen wird.

```

p = {"id": 7, "title": "Zeit"}
① p["lang"] = "de"
   p["title"] = "Die Zeit"
② del p["id"]
   print(p)
③ print("lang" in p, "de" in p)
④ print(p.get("author", "unbekannt"))
⑤ for key, value in p.items():
    print(key, "->", value)

```

- ① eine Zuweisung an einen neuen Schlüssel legt einen Eintrag an, an einen vorhandenen Schlüssel ersetzt sie den Wert
- ② `del` entfernt einen Eintrag
- ③ `in` prüft die Schlüssel, nicht die Werte
- ④ `get` liefert bei einem fehlenden Schlüssel den Ersatzwert, der Zugriff mit eckigen Klammern würde mit einem `KeyError` abbrechen
- ⑤ `items()` liefert Paare aus Schlüssel und Wert, die sich in der Schleife auspacken lassen

```

{'title': 'Die Zeit', 'lang': 'de'}
True False
unbekannt
title -> Die Zeit
lang -> de

```

Drei Eigenschaften sollten Sie kennen. Erstens bricht `p["author"]` bei einem fehlenden Schlüssel mit `KeyError`: `'author'` ab. Wenn ein Schlüssel fehlen kann, fragen Sie vorher mit `in` oder nutzen `get`. Zweitens behält ein Dict die Reihenfolge, in der die Einträge angelegt wurden. Das ist seit Python 3.7 zugesichert. Drittens müssen Schlüssel hashbar sein. Für unsere Beispiele eignen sich Zahlen und Strings, Listen und Dicts eignen sich nicht. Was hashbar bedeutet, erklärt das nächste Kapitel.

Falls Sie in C++ mit `std::map` gearbeitet haben, sind zwei Unterschiede wichtig: Dort sind die Einträge nach dem Schlüssel geordnet, und der Zugriff `m[key]` legt bei einem fehlenden Schlüssel stillschweigend einen Eintrag an.

Exkurs: Brauche ich immer die neueste Python-Version?

NICHT PRÜFUNGSRELEVANT

Nein. Sie brauchen eine Version, die noch gepflegt wird und die alle Bibliotheken unterstützen, die Sie verwenden. Für diesen Kurs ist das die Version aus dem Begleitdokument zur Entwicklungsumgebung, auf Colab die dort installierte Version.

Der Satz „seit Python 3.7 zugesichert“ zeigt, dass sich das Verhalten der Sprache zwischen Versionen ändern kann. Einige Fakten zur Einordnung, Stand Oktober 2026:

- **Wie Python veröffentlicht wird.** Jedes Jahr im Oktober erscheint eine neue Version: 2024 war es 3.13, 2025 war es 3.14. Jede Version wird fünf Jahre gepflegt. In den ersten rund zwei Jahren gibt es Fehlerkorrekturen, danach nur noch Sicherheitskorrekturen.
- **Warum Sie nicht sofort umsteigen müssen.** Bibliotheken wie NumPy und PyTorch stellen für jede Python-Version eigene Pakete bereit. Für eine neue Version liegen diese Pakete erst nach einiger Zeit vor. Teams legen außerdem eine Version fest, damit ein Programm bei allen Beteiligten gleich läuft.
- **Warum Sie nicht zu lange warten sollten.** Nach dem Ende der Pflege gibt es keine Sicherheitskorrekturen mehr, und Bibliotheken stellen die Unterstützung ein. Ein Beispiel: Die Pflege von Python 3.10 endet im Oktober 2026, und PyTorch hat angekündigt, ab Version 2.15 keine Pakete für Python 3.10 mehr bereitzustellen.
- **Was sich zwischen Versionen ändert.** Zwei Beispiele aus diesem Notebook: Die Einfügereihenfolge von Dicts ist seit 3.7 zugesichert, und `zip(..., strict=True)` gibt es seit 3.10. Die Dokumentation vermerkt solche Stellen mit „Neu in Version“ und „Geändert in Version“.
- **Wie Sie nachsehen.** Die nächste Zelle zeigt die Version, mit der Ihr Notebook gerade läuft. Den Stand aller Versionen führt die Seite [Status of Python versions](#).

```

import sys
print(sys.version)      # die Python-Version, mit der dieses Notebook läuft

```

Verschachtelte Daten. Die Kursdaten sind verschachtelt: `passages` ist eine Liste, jedes Element ist ein Dict, und dessen Werte sind Zahlen und Strings. Ein Zugriff wie `passages[0]["title"]` besteht deshalb aus mehreren Schritten. Wenn eine solche Zeile unklar ist, hilft es, sie Schritt für Schritt zu lesen.

Überblick: Wie lese ich einen Datenzugriff?

Lesen Sie von links nach rechts und benennen Sie nach jedem Schritt den Typ des Zwischenergebnisses. Der Typ entscheidet, was als Nächstes erlaubt ist: ein Index bei einer Liste, ein Schlüssel bei einem Dict, eine Methode bei einem String.

Schritt	Ausdruck	Ergebnis	Typ
1	<code>passages</code>	alle Absätze	Liste
2	<code>passages[0]</code>	der erste Absatz	Dict
3	<code>passages[0]["title"]</code>	sein Titel, "Aufzugsanlage"	String
4	<code>passages[0]["title"].lower()</code>	der Titel in Kleinbuchstaben	String

Dasselbe Vorgehen funktioniert, wenn ein Zugriff im anderen steckt. In `passages[queries[0]["expected"]]["title"]` lesen Sie zuerst den inneren Teil: `queries[0]` ist ein Dict, `queries[0]["expected"]` ist die Zahl 108. Damit bleibt `passages[108]["title"]`. Wenn Sie unsicher sind, lassen Sie sich ein Zwischenergebnis mit `type(...)` anzeigen.

Aufgabe 2.3 #2

5 min

Die Vorgabe in der Zelle darunter legt `record` an, eine Kopie des Absatzes 108. Arbeiten Sie nur mit `record`, die Kursdaten in `passages` bleiben unverändert.

1. Speichern Sie den Titel in `title` und die Anzahl der Zeichen des Textes in `n_chars`. Gemeint ist der Text des Absatzes unter dem Schlüssel `"text"`, nicht der Titel.
2. Ergänzen Sie `record` um den Schlüssel `"chars"` mit diesem Wert.
3. Speichern Sie in `has_author`, ob `record` den Schlüssel `"author"` besitzt.

Zusatzaufgabe: Geben Sie in der Zelle nach der Prü fzelle mit einer Schleife über `record.items()` zu jedem Schlüssel den Typ des Werts aus, zum Beispiel `title <class 'str'>`.

Hinweis: `len(...)` liefert bei einem String die Anzahl der Zeichen. Ob ein Schlüssel vorhanden ist, prüft `in`.

[→ Lösung im Anhang](#)

```
record = passages[108].copy()      # Vorgabe: eine eigene Kopie des Dicts, wie copy() bei einer Liste
# TODO: Ihre Lösung hier
...
```

```
# Prü fzelle – einfach ausführen
assert title == "San Diego", "der Titel des Absatzes 108"
assert n_chars == len(passages[108]["text"]), "die Anzahl der Zeichen des Textes"
assert record["chars"] == n_chars, "record bekommt den Schlüssel chars"
assert has_author == False, "einen Schlüssel author gibt es nicht"
assert "chars" not in passages[108], "die Kursdaten bleiben unverändert: nur record ergänzen"
print("Alle Prüfungen bestanden ✓")
```

```
# Zusatzaufgabe: zu jedem Schlüssel von record den Typ des Werts ausgeben
# TODO: Ihre Lösung hier
...
```

Vom Suchen zum Nachschlagen. In `passages` stimmen Position und Nummer überein: `passages[108]` hat die `id` 108. In einer Auswahl gilt das nicht mehr. In `sample` steht der Absatz 108 an Position 1, und `sample[108]` bricht mit einem `IndexError` ab. Um dort einen Absatz über seine Nummer zu finden, müssten Sie die Auswahl durchsuchen. Ein Dict von der Nummer zum Absatz macht daraus ein Nachschlagen.

Aufgabe 2.3 #3

5 min

Schreiben Sie die Funktion `index_by_id(records)`. Sie bekommt eine Liste von Absätzen und gibt ein Dict zurück, das jeder `id` den zugehörigen Absatz zuordnet, also das ganze Dict des Absatzes. Probieren Sie Ihre Funktion zuerst an `sample` aus und sehen Sie sich das Ergebnis an.

Hinweis: Beginnen Sie mit einem leeren Dict `result = {}`. In der Schleife über `records` legt `result[p["id"]] = p` je Absatz einen Eintrag an.

[→ Lösung im Anhang](#)

```
def index_by_id(records):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfwerte – einfach ausführen
_by_id = index_by_id(sample)
assert _by_id is not None, "index_by_id gibt noch nichts zurück (return vergessen?)"
assert len(_by_id) == 3, "ein Eintrag je Absatz der Auswahl"
assert 108 in _by_id, "die Schlüssel sind die Nummern (id), nicht die Positionen in der Liste"
assert _by_id[108]["title"] == "San Diego", "die Nummer 108 führt zum Absatz San Diego"
assert _by_id[78]["title"] == "Mexiko-Stadt"
assert len(index_by_id(passages)) == 400, "auf allen Daten: 400 Einträge"
assert index_by_id([]) == {}, "eine leere Liste ergibt ein leeres Dict"
print("Alle Prüfungen bestanden ✓")
```

Verschachtelte Ergebnisse ansehen. `print` schreibt ein Dict in eine einzige lange Zeile. Bei verschachtelten Daten ist das schwer zu lesen. Die Funktion `pprint` aus dem gleichnamigen Modul verteilt die Einträge auf mehrere Zeilen, sobald sie nicht mehr in eine Zeile passen. Der Name steht für „pretty print“.

```
from pprint import pprint

groups_demo = {"Mexiko-Stadt": [6, 78, 85, 209, 220, 296], "San Diego": [108],
               "Bern": [11, 42, 73, 106, 218], "Aufklärung": [82, 189, 250, 318, 346, 361]}

print(groups_demo)
pprint(groups_demo)

{'Mexiko-Stadt': [6, 78, 85, 209, 220, 296], 'San Diego': [108], 'Bern': [11, 42, 73, 106, 218],
'Aufklärung': [82, 189, 250, 318, 346, 361]}
{'Aufklärung': [82, 189, 250, 318, 346, 361],
'Bern': [11, 42, 73, 106, 218],
'Mexiko-Stadt': [6, 78, 85, 209, 220, 296],
'San Diego': [108]}
```

`pprint` schreibt je Eintrag eine Zeile und ordnet die Schlüssel für die Anzeige alphabetisch. Das Dict selbst ändert sich dadurch nicht. Mit `pprint(groups_demo, sort_dicts=False)` bleibt die Reihenfolge des Dicts auch in der Anzeige erhalten.

 Aufgabe 2.3 #4

10 min

Review. Ein Kollege möchte Absätze über den Titel nachschlagen und legt die Funktion `index_by_title` in der Zelle darunter vor. Die Anforderung lautet: Zu jedem Titel sollen **alle** Absätze des Artikels auffindbar sein.

1. Führen Sie die Zelle aus und wenden Sie die Funktion in einer neuen Zelle auf `sample` an. Vergleichen Sie die Anzahl der Einträge mit der Anzahl der Absätze.
2. Notieren Sie Ihr Urteil als Kommentar: Erfüllt die Funktion die Anforderung?
3. Schreiben Sie `group_by_title(records)`. Sie gibt ein Dict zurück, das jedem Titel die Liste der Nummern (`id`) aller Absätze mit diesem Titel zuordnet. Abgelegt werden nur die Nummern, nicht die ganzen Absätze wie bei `index_by_id`. Die Nummern stehen in der Reihenfolge, in der die Absätze in `records` vorkommen. Das Ergebnis ist aufgebaut wie `groups_demo` in der Zelle oben. Sehen Sie es sich für `sample` mit `pprint` an.

Zusatzaufgabe: Bestimmen Sie in der Zelle nach der Prü fzelle mit einer Schleife über `group_by_title(passages).items()`, welcher Titel die meisten Absätze hat und wie viele es sind.

Hinweis: Beim ersten Absatz eines Titels gibt es die Liste noch nicht. Prü fen Sie mit `if title not in result:`, ob der Titel schon ein Schlüssel ist, und legen Sie sonst mit `result[title] = []` eine leere Liste an. Danach hängt `result[title].append(...)` die Nummer an.

[→ Lösung im Anhang](#)

```
# Die zu prüfende Fassung
def index_by_title(records):
    result = {}
    for p in records:
        result[p["title"]] = p
    return result

# Ihr Urteil als Kommentar:
# TODO: Ihre Lösung hier
...

def group_by_title(records):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prü fzelle – einfach ausführen
_groups = group_by_title(sample)
assert _groups is not None, "group_by_title gibt noch nichts zurück (return vergessen?)"
_expected = {"Mexiko-Stadt": [6, 78], "San Diego": [108]}
assert _groups == _expected, "je Titel die Liste der Nummern, in der Reihenfolge der Absätze"
groups = group_by_title(passages) # wird in späteren Kapiteln wieder gebraucht
assert len(groups) == 217, "auf allen Daten: 217 verschiedene Titel"
_total = 0
for _ids in groups.values():
    _total += len(_ids)
assert _total == 400, "kein Absatz darf verloren gehen"
assert groups["Mexiko-Stadt"] == [6, 78, 85, 209, 220, 296]
assert group_by_title([]) == {}
print("Alle Prüfungen bestanden ✓")
```

```
# Zusatzaufgabe: der Titel mit den meisten Absätzen
# TODO: Ihre Lösung hier
...
```

Das Muster aus `group_by_title` heißt Gruppieren: Ein Dict ordnet jedem Schlüssel eine Liste zu, und jeder Datensatz landet in der Liste seines Schlüssels. Es ist eines der häufigsten Muster in der Datenverarbeitung. Die Prü fzelle hat das Ergebnis für alle Absätze in `groups` abgelegt, das Sortier-Kapitel verwendet es wieder.

Überblick: Der Dict-Werkzeugkasten

Mit Zugriff, Zuweisung, `in`, `get` und `items` kommen Sie durch diesen Kursteil. Der volle Werkzeugkasten steht hier zum Nachschlagen. Dabei steht `d` für ein Dict.

Werkzeug	Tut was	Gibt zurück
<code>d[key]</code>	liest den Wert zum Schlüssel	den Wert, bei fehlendem Schlüssel <code>KeyError</code>
<code>d.get(key, default)</code>	liest den Wert oder den Ersatzwert	den Wert oder <code>default</code>
<code>key in d</code>	prüft, ob der Schlüssel vorhanden ist	<code>bool</code>
<code>d[key] = value</code>	legt den Eintrag an oder ersetzt den Wert	nichts, verändert <code>d</code>
<code>del d[key]</code>	entfernt den Eintrag	nichts, verändert <code>d</code>
<code>d.pop(key)</code>	entfernt den Eintrag	den Wert, verändert <code>d</code>
<code>len(d)</code>	Anzahl der Einträge	<code>int</code>
<code>d.keys()</code> , <code>d.values()</code> , <code>d.items()</code>	Schlüssel, Werte, Paare zum Durchlaufen	eine Sicht auf das Dict
<code>d.update(other)</code>	übernimmt alle Einträge aus <code>other</code>	<code>None</code> , verändert <code>d</code>
<code>d.copy()</code>	flache Kopie, erklärt im Kapitel 2.5	neues Dict

Lernfragen 2.3 #1

1. Was gibt `len(index_by_title(passages))` zurück, und warum ist es weniger als 400?
2. Welche Aussagen über Dicts stimmen?
 - `"title" in p` prüft, ob es den Schlüssel `"title"` gibt.
 - `p["author"]` liefert `None`, wenn der Schlüssel fehlt.
 - Eine Zuweisung an einen vorhandenen Schlüssel ersetzt den Wert.
 - Ein Dict darf jeden Wert als Schlüssel verwenden.

[→ Lösung im Anhang.](#)

Zum Nachschlagen: [Python-Tutorial 5.3. Tupel und Sequenzen](#) und [5.5. Dictionaries](#).

2.4 Text verarbeiten, zählen, Mengen bilden

Dieses Kapitel baut eine kleine Verarbeitungskette: Aus einem Text werden Wörter, die Wörter werden gezählt, und aus allen Wörtern entsteht das Vokabular. Den Anfang machen die String-Methoden, die dafür nötig sind.

```

text = " Die GPU rechnet. Die CPU wartet. "
① print(text.strip())
② print(text.lower())
③ words = text.split()
   print(words)
④ print(" | ".join(words))
⑤ print(text.replace("GPU", "Grafikkarte"))
⑥ print("GPU" in text, "TPU" in text)
⑦ print("rechnet.".strip(".,;!?"))

```

- ① `strip()` entfernt Leerraum an beiden Enden
- ② `lower()` liefert den Text in Kleinbuchstaben
- ③ `split()` trennt an Leerraum und liefert eine Liste von Wörtern
- ④ `join` setzt eine Liste von Strings zusammen, der String davor ist das Trennzeichen
- ⑤ `replace` ersetzt jedes Vorkommen eines Teilstrings
- ⑥ `in` prüft bei Strings, ob ein Teilstring vorkommt
- ⑦ `strip` mit einem Argument entfernt an beiden Enden alle Zeichen, die im Argument vorkommen

```

Die GPU rechnet. Die CPU wartet.
die gpu rechnet. die cpu wartet.
['Die', 'GPU', 'rechnet.', 'Die', 'CPU', 'wartet.']
Die | GPU | rechnet. | Die | CPU | wartet.
Die Grafikkarte rechnet. Die CPU wartet.
True False
rechnet

```

Strings sind unveränderlich. Keine dieser Methoden verändert den String, auf dem sie aufgerufen wird. Jede liefert ihr Ergebnis zurück: `split()` eine Liste von Strings, die anderen hier gezeigten Methoden einen neuen String. Am Ende der Zelle oben enthält `text` noch die Großbuchstaben und den Leerraum. Um das Ergebnis zu behalten, weisen Sie es zu: `text = text.lower()`.

Bei Listen ist es umgekehrt: Methoden wie `append` verändern die Liste und geben `None` zurück. Das war die `None`-Falle aus Teil 1. Der Unterschied folgt aus den beiden Typen. Ein String lässt sich nicht verändern. Eine String-Methode kann den String deshalb nicht umformen, sie gibt ihr Ergebnis zurück: `lower()` einen neuen String, `split()` eine Liste. Eine Liste lässt sich verändern. Für Methoden, die ihr Objekt nur verändern, gilt in Python die Konvention, `None` zurückzugeben. An der Rückgabe ist so zu erkennen, dass die Methode das vorhandene Objekt verändert hat und kein neues liefert.

Aufgabe 2.4 #1

2 min

Der Code in der Zelle darunter soll die Anfrage in Kleinbuchstaben umwandeln, das Fragezeichen entfernen und in Wörter zerlegen. Das erwartete Ergebnis ist `['wo', 'liegt', 'san', 'diego']`. Führen Sie die Zelle aus, vergleichen Sie und korrigieren Sie den Fehler direkt in der Zelle.

[→ Lösung im Anhang](#)

```

query = "Wo liegt San Diego?"
query.lower()
query.strip("?")
query_words = query.split()
print(query_words)

```

```

# Prüfwerte – einfach ausführen
_expected = ["wo", "liegt", "san", "diego"]
assert query_words == _expected, "String-Methoden verändern den String nicht: das Ergebnis zuweisen"
print("Alle Prüfungen bestanden ✓")

```

Überblick: Wie finde ich heraus, was ein Objekt kann?

Sie müssen die Methoden nicht auswendig kennen. Drei Werkzeuge im Notebook zeigen Ihnen, was ein Objekt anbietet und was eine Methode tut.

- **Tabulatortaste.** Tippen Sie `text.` und drücken Sie die Tabulatortaste. Das Notebook zeigt alle Methoden des Objekts. Dafür muss die Zelle, die `text` anlegt, ausgeführt sein.
- **`help(...)`.** Der Aufruf `help(str.split)` zeigt die Beschreibung der Methode `split` für Strings. Für eine Liste lautet der Name `list`, für ein Dict `dict`.
- **Fragezeichen.** Im Notebook zeigt `str.split?` dieselbe Beschreibung in Kurzform.

Die Beschreibung, die `help` und das Fragezeichen anzeigen, heißt Docstring. Das ist ein String, der als erste Anweisung im Rumpf einer Funktion steht und sagt, was sie tut. Üblich sind drei Anführungszeichen, damit kann der Text über mehrere Zeilen gehen. Die Anzeige des Fragezeichens nennt ihn hinter dem Wort `Docstring`. Auch eigene Funktionen können einen Docstring bekommen, die zweite Zelle unten zeigt ein Beispiel.

Die vollständige Beschreibung aller Typen steht in der [Dokumentation der eingebauten Typen](#).

```
help(str.strip)      # die Beschreibung der String-Methode strip
```

```
def first_word(text):
    """Gibt das erste Wort des Textes zurück."""
    return text.split()[0]
```

```
help(first_word)     # zeigt den Docstring der eigenen Funktion
```

Help on function first_word in module __main__:

```
first_word(text)
    Gibt das erste Wort des Textes zurück.
```

Überblick: Der String-Werkzeugkasten

Die Methoden vom Anfang dieses Kapitels und einige weitere zum Nachschlagen. Dabei steht `s` für einen String. Keine der Methoden verändert `s`.

Werkzeug	Tut was	Gibt zurück
<code>s.lower()</code> , <code>s.upper()</code>	Klein- oder Großbuchstaben	neuer String
<code>s.strip()</code> , <code>s.strip(chars)</code>	entfernt Leerraum oder die Zeichen aus <code>chars</code> an beiden Enden	neuer String
<code>s.split()</code> , <code>s.split(sep)</code>	trennt an Leerraum oder am Trennzeichen <code>sep</code>	Liste von Strings
<code>sep.join(words)</code>	setzt eine Liste von Strings mit <code>sep</code> zusammen	neuer String
<code>s.replace(old, new)</code>	ersetzt jedes Vorkommen von <code>old</code>	neuer String
<code>s.startswith(x)</code> , <code>s.endswith(x)</code>	prüft Anfang oder Ende	<code>bool</code>
<code>x in s</code>	prüft, ob <code>x</code> als Teilstring vorkommt	<code>bool</code>
<code>s.find(x)</code>	sucht <code>x</code>	Position als <code>int</code> , <code>-1</code> wenn <code>x</code> fehlt
<code>s.count(x)</code>	zählt die Vorkommen von <code>x</code>	<code>int</code>
<code>len(s)</code>	Anzahl der Zeichen	<code>int</code>

 Aufgabe 2.4 #2

10 min

Schreiben Sie die Funktion `clean_words(text)`. Sie gibt die Wörter eines Textes als Liste zurück und erfüllt drei Anforderungen:

1. Alle Wörter stehen in Kleinbuchstaben.
2. An Anfang und Ende jedes Worts sind die Zeichen aus `PUNCTUATION` entfernt.
3. Wörter, von denen danach nichts übrig bleibt, fehlen im Ergebnis.

Entwickeln Sie an kurzen Beispielen wie `"Die GPU, die gpu!"` und wenden Sie die Funktion danach auf `sample[1]["text"]` an.

Zusatzaufgabe: Geben Sie in der Zelle nach der Prü fzelle `clean_words(passages[108]["text"])` aus. Finden Sie die Wörter, an denen trotz der Bereinigung noch Zeichen hängen, die keine Buchstaben sind. Woran liegt das?

Hinweis: Zerlegen Sie den Text erst mit `lower()` und `split()`. Bereinigen Sie dann in einer Schleife jedes Wort mit `strip(PUNCTUATION)` und hängen Sie es nur an die Ergebnisliste an, wenn es nicht der leere String `""` ist.

[→ Lösung im Anhang](#)

```
PUNCTUATION = ".,:;!()?\"'\"" # Vorgabe: Zeichen, die an den Wortenden entfernt werden
```

```
def clean_words(text):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prü fzelle – einfach ausführen
assert clean_words("Die GPU") is not None, "clean_words gibt noch nichts zurück (return vergessen?)"
_expected = ["die", "gpu", "die", "gpu"]
assert clean_words("Die GPU, die gpu!") == _expected, "Kleinbuchstaben, ohne Satzzeichen am Wortende"
assert clean_words("(GPU)") == ["gpu"], "Satzzeichen an beiden Enden des Worts entfernen"
assert clean_words("Halt ... stopp") == ["halt", "stopp"], "ein leeres Wort fehlt im Ergebnis"
assert clean_words(" ") == [], "ein Text ohne Wörter ergibt eine leere Liste"
assert clean_words("Wo liegt San Diego?") == ["wo", "liegt", "san", "diego"]
assert clean_words("E-Mail") == ["e-mail"], "Zeichen im Wortinneren bleiben stehen"
assert len(clean_words(passages[108]["text"])) > 50, "auf einem Absatz der Kursdaten"
print("Alle Prüfungen bestanden ✓")
```

```
# Zusatzaufgabe: An welchen Wörtern hängen noch Zeichen, die keine Buchstaben sind?
# TODO: Ihre Lösung hier
...
```

 Kurz überlegen

Die nächste Zelle sammelt die Wörter zweier Texte in einer Liste. Was gibt sie aus?

```
all_words = []
all_words.append(clean_words("Die GPU"))
all_words.append(clean_words("Die CPU wartet"))
print(len(all_words))
```

append oder extend? Diese und die folgenden Zellen verwenden Ihre Funktion `clean_words`. `append` hängt sein Argument als ein einziges Element an. Ist das Argument eine Liste, entsteht eine Liste in der Liste. `extend` hängt dagegen jedes Element des Arguments einzeln an. Beim Sammeln von Wörtern über mehrere Texte ist `extend` das passende Werkzeug. Wie `append` verändert `extend` die Liste und gibt `None` zurück.

```
all_words = []
for p in sample:
    all_words.extend(clean_words(p["text"]))    # die Wörter jedes Absatzes einzeln anhängen
print(len(all_words), "Wörter in den drei Absätzen der Auswahl")
print(all_words[:8])
```

```
427 Wörter in den drei Absätzen der Auswahl
['museo', 'frida', 'kahlo', 'das', 'haus', 'von', 'frida', 'kahlo']
```

Zählen. Wie oft kommt jedes Wort vor? Das Muster dafür ist ein Dict, das jedem Wort seine Anzahl zuordnet. Beim ersten Vorkommen gibt es den Eintrag noch nicht. Die Zeile `counts[word] += 1` allein bricht deshalb mit `KeyError: 'die'` ab, denn sie liest erst den alten Wert. Die nächste Zelle zeigt zwei Fassungen, die das berücksichtigen.

```
counts = {}
for word in ["die", "gpu", "die"]:
    if word in counts:
        counts[word] += 1    # das Wort gab es schon: eins dazu
    else:
        counts[word] = 1    # erstes Vorkommen: Eintrag anlegen
print(counts)

counts = {}
for word in ["die", "gpu", "die"]:
    ① counts[word] = counts.get(word, 0) + 1
print(counts)
```

① `get(word, 0)` liefert die bisherige Anzahl oder 0, wenn das Wort noch fehlt

```
{'die': 2, 'gpu': 1}
{'die': 2, 'gpu': 1}
```

Aufgabe 2.4 #3

2 min

Schreiben Sie die Funktion `count_words(words)`. Sie bekommt eine Liste von Wörtern und gibt ein Dict zurück, das jedem Wort die Anzahl seiner Vorkommen zuordnet. Die Zelle darunter wendet Ihre Funktion danach auf die Wörter aller 400 Absätze an.

[→ Lösung im Anhang](#)

```
def count_words(words):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfwerte – einfach ausführen
assert count_words(["die"]) is not None, "count_words gibt noch nichts zurück (return vergessen?)"
assert count_words(["die", "gpu", "die"]) == {"die": 2, "gpu": 1}, "die Anzahl je Wort"
assert count_words([]) == {}, "eine leere Liste ergibt ein leeres Dict"
_words = ["a", "b", "a"]
count_words(_words)
assert _words == ["a", "b", "a"], "die übergebene Liste bleibt unverändert"
print("Alle Prüfungen bestanden ✓")
```

```

corpus_words = []
for p in passages:
    corpus_words.extend(clean_words(p["text"]))    # die Wörter aller 400 Absätze
word_counts = count_words(corpus_words)
print(len(corpus_words), "Wörter,", len(word_counts), "verschiedene")
print("gpu:", word_counts.get("gpu", 0), " stadt:", word_counts.get("stadt", 0))
print("der:", word_counts.get("der", 0))

```

```

59498 Wörter, 16840 verschiedene
gpu: 0 stadt: 49
der: 2337

```

Die häufigsten Wörter lassen sich aus `word_counts` noch nicht bequem ablesen. Dafür fehlt das Sortieren, das im Kapitel [2.7](#) folgt.

Sets. Ein Set enthält jeden Wert höchstens einmal. Im Termin haben Sie damit die verschiedenen Titel gezählt und gemessen, dass die Prüfung mit `in` bei einem Set im Mittel nicht von der Größe abhängt. Die nächste Zelle zeigt, was Sets darüber hinaus können.

```

a = set(clean_words("Die GPU rechnet, die CPU wartet."))
b = set(clean_words("Die GPU wartet nicht."))
print(a)
① print(a & b)
② print(a | b)
③ print(a - b)
④ a.add("gpu")
print(len(a))
⑤ print(type(set()), type({}))

```

- ① `&` bildet die Schnittmenge: Werte, die in beiden Sets vorkommen
- ② `|` bildet die Vereinigung: Werte, die in mindestens einem der Sets vorkommen
- ③ `-` bildet die Differenz: Werte aus dem ersten Set, die im zweiten fehlen
- ④ `add` fügt einen Wert hinzu; ist er schon enthalten, ändert sich nichts
- ⑤ das leere Set heißt `set()`, denn `{}` ist ein leeres Dict

```

{'rechnet', 'die', 'wartet', 'gpu', 'cpu'}
{'wartet', 'gpu', 'die'}
{'rechnet', 'die', 'wartet', 'nicht', 'gpu', 'cpu'}
{'rechnet', 'cpu'}
5
<class 'set'> <class 'dict'>

```

Ein Set hat keine Reihenfolge. Die Ausgabe oben kann bei Ihnen anders geordnet sein als im Skript, und ein Zugriff wie `a[0]` ist nicht möglich.

Hashbar. Die Elemente eines Sets und die Schlüssel eines Dicts müssen hashbar sein. Dicts und Sets finden einen Wert über seinen Hashwert. Das ist eine ganze Zahl, die Python aus dem Wert berechnet. Im Termin haben Sie gesehen, dass ein Set mit dem Hashwert direkt an der zugehörigen Stelle nachsieht. Das funktioniert nur, wenn sich der Hashwert nicht ändert, solange der Wert im Set oder Dict steckt. Listen, Dicts und Sets lassen sich verändern, sie sind deshalb nicht hashbar.

Hashbar sind Zahlen, Strings und Tupel, deren Elemente selbst hashbar sind. Die eingebaute Funktion `hash` zeigt den Unterschied. Probieren Sie es in der nächsten Zelle aus.

```

print(hash("gpu"))          # ein String ist hashbar: das Ergebnis ist eine ganze Zahl
print(hash(("Gehirn", 0.89))) # ein Tupel aus hashbaren Werten ist es auch

# Entfernen Sie nacheinander das Zeichen # vor einer der nächsten Zeilen und führen Sie die Zelle aus:
# hash(["die", "gpu"])      # TypeError: unhashable type: 'list'
# [{"die", "gpu"}: 1]      # derselbe Fehler: Eine Liste kann kein Schlüssel sein
# hash(("Gehirn", [1, 2])) # derselbe Fehler: Das Tupel enthält eine Liste

```

Die Zahlen für Strings unterscheiden sich von Sitzung zu Sitzung, das ist beabsichtigt. Das letzte Beispiel zeigt, dass „unveränderlich“ als Kriterium nicht genügt: Das Tupel selbst lässt sich nicht verändern, die Liste darin schon. Damit ist auch das Tupel nicht hashbar. Die genaue Definition steht im [Glossar der Python-Dokumentation](#).

Aufgabe 2.4 #4

10 min

Schreiben Sie die Funktion `shared_words(text_a, text_b)`. Sie gibt das Set der Wörter zurück, die in beiden Texten vorkommen. Verwenden Sie `clean_words`, damit Großschreibung und Satzzeichen keine Rolle spielen.

Bestimmen Sie danach das Vokabular des Korpus: Legen Sie die neue Variable `corpus_vocabulary` an, das Set aller verschiedenen Wörter in `corpus_words`. Die Liste `corpus_words` mit den Wörtern aller 400 Absätze stammt aus der Zelle nach der Aufgabe zu `count_words`.

Zusatzaufgabe: Die Anfrage 0 lautet „Wo liegt San Diego?“. Berechnen Sie in der Zelle nach der Prüfzelle `shared_words(queries[0][\"text\"], passages[108][\"text\"])` und `shared_words(queries[0][\"text\"], passages[6][\"text\"])`. Der erste Absatz ist der gesuchte, der zweite handelt von einem Museum in Mexiko-Stadt. Wie viele Wörter teilt die Anfrage mit jedem der beiden Absätze? Suchen Sie im Text des Absatzes 6 die Stellen, an denen die gemeinsamen Wörter vorkommen.

Hinweis: `set(...)` macht aus einer Liste ein Set. Die Wörter, die in beiden Sets vorkommen, liefert der Operator `&`.

[→ Lösung im Anhang](#)

```
def shared_words(text_a, text_b):
    # TODO: Ihre Lösung hier
    ...

# das Set aller verschiedenen Wörter in corpus_words
# TODO: Ihre Lösung hier
...
```

```
# Prüfzelle – einfach ausführen
assert shared_words("a", "a") is not None, "shared_words gibt noch nichts zurück (return vergessen?)"
_shared = shared_words("Die GPU rechnet.", "Rechnet die CPU?")
assert _shared == {"die", "rechnet"}, "Großschreibung und Satzzeichen dürfen keine Rolle spielen"
assert shared_words("GPU", "CPU") == set(), "ohne gemeinsame Wörter: das leere Set"
assert type(shared_words("a b", "b c")) == set, "das Ergebnis ist ein Set"
assert shared_words(queries[0][\"text\"], passages[108][\"text\"] == {\"liegt\", \"san\", \"diego\"}
assert type(corpus_vocabulary) == set, "corpus_vocabulary ist ein Set"
assert corpus_vocabulary == set(corpus_words), "jedes Wort aus corpus_words genau einmal"
print("Alle Prüfungen bestanden ✓")
```

```
# Zusatzaufgabe: Wie viele Wörter teilt die Anfrage 0 mit den Absätzen 108 und 6?
# TODO: Ihre Lösung hier
...
```

Stichwörter gegen Bedeutung. Die gemeinsamen Wörter ergeben eine einfache Stichwortsuche: Der Absatz, der die meisten Wörter mit der Anfrage teilt, gilt als bester Treffer. Die nächste Zelle prüft für die acht Beispielanfragen, wie oft der erwartete Absatz dabei allein vorn liegt. Sie rechnet zweimal: einmal mit `clean_words` und einmal nur mit `lower().split()`, also ohne die Satzzeichen zu entfernen.

```

def to_word_set(text, clean):
    if clean:
        return set(clean_words(text))          # Satzzeichen an den Wortenden entfernt
    return set(text.lower().split())           # nur zerlegt

def count_alone_on_top(clean):
    passage_sets = []
    for p in passages:
        passage_sets.append(to_word_set(p["text"], clean))
    alone = 0
    for q in queries:
        query_set = to_word_set(q["text"], clean)
        overlaps = []                          # je Absatz die Anzahl gemeinsamer Wörter
        for words in passage_sets:
            overlaps.append(len(query_set & words))
        best = max(overlaps)
        if overlaps[q["expected"]] == best and overlaps.count(best) == 1:
            alone += 1                          # der erwartete Absatz liegt allein vorn
    return alone

print("mit clean_words:      ", count_alone_on_top(True), "von 8")
print("nur lower() und split():", count_alone_on_top(False), "von 8")
q4 = queries[4]
print("Anfrage 4 und ihr Absatz teilen:", shared_words(q4["text"], passages[q4["expected"]]["text"]))

```

```

mit clean_words:      4 von 8
nur lower() und split(): 2 von 8
Anfrage 4 und ihr Absatz teilen: set()

```

Die Stichwortsuche findet den erwarteten Absatz nur bei einem Teil der Anfragen, und das Ergebnis hängt von der Vorverarbeitung ab. Bei der Anfrage 0 („Wo liegt San Diego?“) teilt ein Absatz über ein Museum in Mexiko-Stadt ebenso viele Wörter mit der Anfrage wie der gesuchte Absatz, denn dort kommen „Diego Rivera“ und „San Angel“ vor. Die Anfrage 4 („Auf was beruhen Browserdatenstatistiken?“) teilt mit ihrem Absatz kein einziges Wort. Die semantische Suche aus dem Termin hat ihn trotzdem auf Platz 1 gefunden, weil sie Bedeutungen vergleicht und keine Zeichenfolgen.

Die Bereinigung in `clean_words` ist bewusst einfach. Zeichen wie der Apostroph oder der Gedankenstrich bleiben hängen, und „Stadt“ und „Städte“ gelten als verschiedene Wörter. Für das Zerlegen von Texten gibt es mächtigere Werkzeuge, etwa reguläre Ausdrücke. Sprachmodelle zerlegen Texte mit eigenen Verfahren in Einheiten, die Tokens heißen.

Lernfragen 2.4 #2

1. Warum liefert die Stichwortsuche mit `clean_words` mehr richtige Treffer als mit `lower().split()` allein?
2. Was ist der Unterschied zwischen `{}` und `set()`?
3. Sie haben eine Liste mit 100.000 Wörtern und wollen für viele einzelne Wörter prüfen, ob sie darin vorkommen. Welchen Container wählen Sie für die Prüfung, und warum?

[→ Lösung im Anhang](#)

Zum Nachschlagen: [Python-Tutorial 5.4. Mengen](#) und die [String-Methoden in der Dokumentation](#).

2.5 Slicing und Kopien

Ein Slice schneidet einen Ausschnitt aus einer Liste, einem Tupel oder einem String. Die Grundform `xs[:2]` kennen Sie aus Teil 1. Die volle Schreibweise ist `xs[start:stop:step]`. Jede der drei Angaben darf fehlen.

```

xs = [0, 10, 20, 30, 40, 50, 60]
① print(xs[2:5])
② print(xs[:3], xs[3:])
③ print(xs[-2:], xs[:-2])
④ print(xs[:2])
⑤ print(xs[::-1])
⑥ print("Tensor"[:3], ("Gehirn", 0.89, 92)[1:])

```

- ① start gehört dazu, stop ist ausgeschlossen
- ② fehlt start, beginnt der Slice am Anfang; fehlt stop, reicht er bis zum Ende
- ③ negative Angaben zählen vom Ende
- ④ step ist die Schrittweite
- ⑤ eine negative Schrittweite läuft rückwärts
- ⑥ Slices funktionieren genauso auf Strings und Tupeln

```

[20, 30, 40]
[0, 10, 20] [30, 40, 50, 60]
[50, 60] [0, 10, 20, 30, 40]
[0, 20, 40, 60]
[60, 50, 40, 30, 20, 10, 0]
Ten (0.89, 92)

```

Warum ist das Ende ausgeschlossen? Die Regel hat zwei praktische Folgen. `xs[:3]` und `xs[3:]` teilen eine Liste an der Position 3, ohne dass ein Element doppelt vorkommt oder fehlt. Und die Anzahl der Elemente in `xs[2:5]` ist `5 - 2`. Dieselbe Regel gilt für `range`.

Ein Slice über das Ende hinaus bricht nicht ab. `xs[5:100]` liefert die vorhandenen Elemente ab Position 5, und `xs[100:]` liefert eine leere Liste. Ein einzelner Index ist strenger: `xs[100]` bricht mit `IndexError: list index out of range` ab.

Kurz überlegen

Ein Slice einer Liste ist eine neue Liste. Die nächste Zelle legt eine Auswahl aus drei Datensätzen an, entfernt mit `pop()` ein Element aus der Auswahl und ändert einen Titel. `pop()` steht im Listen-Werkzeugkasten von Teil 1: Die Methode entfernt das letzte Element einer Liste und gibt es zurück. Was gibt die Zelle aus?

```

records = [{"title": "Zeit"}, {"title": "Koran"}, {"title": "Gehirn"}]
selection = records[:2]
selection.pop()           # entfernt das letzte Element aus der Auswahl
selection[0]["title"] = "ZEIT" # ändert einen Datensatz über die Auswahl
print(len(records), records[0]["title"])

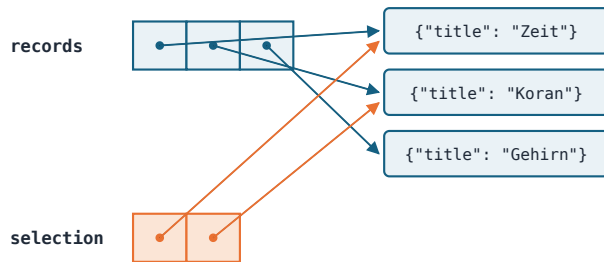
```

Neue Liste heißt nicht unabhängige Daten

Ein Slice, `copy()` und `list(...)` erzeugen eine neue Liste, die auf dieselben Elemente verweist. Das ist die flache Kopie aus Teil 1. Bei Zahlen und Strings macht das keinen Unterschied, denn sie lassen sich nicht verändern. Für Tupel gilt das nur, wenn auch ihre Elemente unveränderlich sind. Sind die Elemente Dicts oder Listen, wirkt eine Änderung an einem Element in beiden Listen. Für unabhängige Daten müssen auch die Elemente kopiert werden.

Flache Kopie

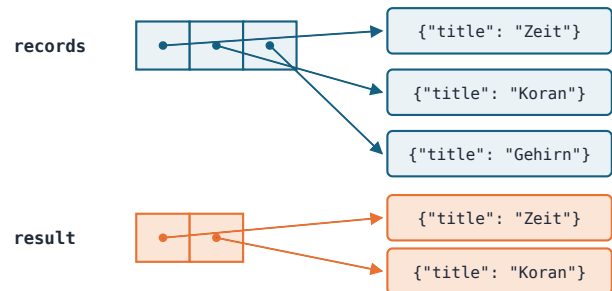
```
selection = records[:2]
```



Zwei Listen, dieselben Dicts. Eine Änderung an einem Dict ist über beide Listen sichtbar.

Elemente kopiert

```
result.append(r.copy()) je Datensatz r
```



Jede Liste hat ihre eigenen Dicts. Eine Änderung im Ergebnis erreicht die Eingabe nicht.

Aufgabe 2.5 #1

10 min

Schreiben Sie die Funktion `copy_first(records, n)`. Sie gibt die ersten `n` Datensätze als neue Liste zurück. Für `n` dürfen Sie eine ganze Zahl ab 0 voraussetzen. Sind weniger als `n` Datensätze vorhanden, gibt die Funktion alle vorhandenen zurück, mit `n = 0` ist das Ergebnis leer. Jeder Datensatz im Ergebnis ist eine eigene Kopie, sodass eine Änderung am Ergebnis die Eingabe nie verändert. Die Datensätze sind Dicts mit Zahlen und Strings als Werten.

Schreiben Sie in die zweite Zelle einen eigenen Test: Er ändert einen Titel im Ergebnis und zeigt mit `assert`, dass der Titel in `test_records` unverändert ist. Arbeiten Sie dafür mit den Beispieldaten `test_records` aus der Vorgabe, nicht mit den Kursdaten.

Hinweis: Laufen Sie über den Slice `records[:n]` und hängen Sie von jedem Datensatz `r` `copy()` an die Ergebnisliste an. `copy()` gibt es bei Dicts wie bei Listen.

[→ Lösung im Anhang](#)

```
def copy_first(records, n):
    # TODO: Ihre Lösung hier
    ...
```

```
# Vorgabe: Beispieldaten für Ihren Test
test_records = [{"id": 1, "title": "Zeit"},
                 {"id": 2, "title": "Koran"},
                 {"id": 3, "title": "Gehirn"}]
```

```
# Ihr Test: einen Titel im Ergebnis ändern, dann zeigen, dass test_records unverändert ist
# TODO: Ihre Lösung hier
...
```

```
# Prü fzelle – einfach ausführen (legt ihre eigenen Beispieldaten an)
_records = [{"id": 1, "title": "Zeit"}, {"id": 2, "title": "Koran"}, {"id": 3, "title": "Gehirn"}]
_part = copy_first(_records, 2)
assert _part is not None, "copy_first gibt noch nichts zurück (return vergessen?)"
assert _part == [{"id": 1, "title": "Zeit"}, {"id": 2, "title": "Koran"}], "die ersten n Datensätze"
assert _part[0] is not _records[0], "jeder Datensatz im Ergebnis ist eine eigene Kopie"
_part[0]["title"] = "ZEIT"
_part.pop()
assert _records[0]["title"] == "Zeit", "eine Änderung am Ergebnis darf die Eingabe nicht verändern"
assert len(_records) == 3, "die Eingabeliste bleibt unverändert"
assert copy_first(_records, 0) == [], "n = 0 ergibt eine leere Liste"
assert len(copy_first(_records, 10)) == 3, "n größer als die Liste: alle Datensätze"
print("Alle Prüfungen bestanden ✓")
```

`copy()` kopiert bei einem Dict nur die oberste Ebene, wie bei einer Liste. Für die Absätze genügt das, denn ihre Werte sind Zahlen und Strings. Stecken in den Datensätzen weitere Listen oder Dicts, kopiert `copy.deepcopy` aus dem Modul `copy` alle Ebenen.

Überblick: Wie übergibt Python Argumente?

Python übergibt einer Funktion den Verweis auf das Objekt. Der Parameter ist ein weiterer Name für dasselbe Objekt, das Objekt wird nicht kopiert. Dieses Verfahren heißt **Call by Sharing**.

Falls Sie aus C++ die Begriffe Call by Value und Call by Reference kennen: Python verhält sich wie keine der beiden Übergabearten in C++.

- **Call by Value** heißt in C++: Die Funktion bekommt eine Kopie des Objekts. In Python wird das Objekt nicht kopiert. `copy_first` muss deshalb selbst kopieren.
- **Call by Reference** heißt in C++: Der Parameter steht für die Variable des Aufrufers, eine Zuweisung an den Parameter ändert diese Variable. In Python lässt eine Zuweisung an den Parameter nur den lokalen Namen auf ein anderes Objekt zeigen.

Das [Python-Tutorial](#) beschreibt dasselbe Verhalten als Call by Value und ergänzt, dass der übergebene Wert immer ein Verweis auf das Objekt ist und nicht das Objekt selbst. Gemeint ist dasselbe wie hier.

Daraus folgen zwei Regeln. Verändert die Funktion das übergebene Objekt, zum Beispiel mit `records.append(...)` oder `records[0]["title"] = ...`, sieht der Aufrufer die Änderung. Weist die Funktion dem Parameter etwas Neues zu, zum Beispiel mit `records = []`, merkt der Aufrufer nichts. Das Beispiel dazu steht in Teil 1 im Kapitel „Variablen sind Etiketten“ (`add_flag` und `replace`).

Für Ihre eigenen Funktionen heißt das: Eine Funktion, die ihre Eingabe nicht verändern soll, baut ihr Ergebnis in einer neuen Liste auf und verändert auch die Elemente der Eingabe nicht. Übernimmt sie Elemente ins Ergebnis, gehören diese danach zu beiden Listen. Soll der Aufrufer das Ergebnis frei verändern können, kopiert die Funktion auch die Elemente, wie `copy_first`.

Call by Value

C++, Übergabe als Wert
 void f(std::vector<int> xs)



Die Funktion bekommt eine eigene Kopie.

Die Funktion verändert das Objekt:
Der Aufrufer merkt nichts.

Die Funktion weist dem Parameter etwas Neues zu:
Der Aufrufer merkt nichts.

Call by Reference

C++, Übergabe als Referenz
 void f(std::vector<int>& xs)



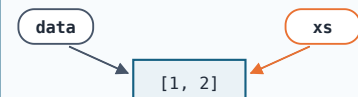
xs steht für die Variable data des Aufrufers.

Die Funktion verändert das Objekt:
Der Aufrufer sieht die Änderung.

Die Funktion weist dem Parameter etwas Neues zu:
data bekommt den neuen Wert.

Call by Sharing

Python
 def f(xs):



Beide Namen verweisen auf dasselbe Objekt.

Die Funktion verändert das Objekt:
Der Aufrufer sieht die Änderung.

Die Funktion weist dem Parameter etwas Neues zu:
Der Aufrufer merkt nichts.

Die Slicing-Schreibweise begegnet Ihnen in der nächsten Lerneinheit bei NumPy wieder. Die Schreibweise ist dort dieselbe, das Verhalten beim Kopieren ist ein anderes. Die Regeln dazu lernen Sie im Termin.

Lernfragen 2.5 #2

1. Welche Aussagen über `xs = [1, 2, 3, 4, 5]` stimmen?

- `xs[:2] + xs[2:]` ergibt wieder `[1, 2, 3, 4, 5]`.
- `xs[1:3]` enthält drei Elemente.
- `xs[10:]` liefert eine leere Liste.
- `ys = xs[:]` bewirkt, dass `ys.append(6)` auch `xs` verlängert.

2. `sample` enthält drei Absätze aus `passages`. Was bewirkt `sample[0]["title"] = "Test"` in den Kursdaten, und warum?

→ [Lösung im Anhang](#).

Zum Nachschlagen: [Python-Tutorial 3.1.3. Listen](#) zum Slicing. Zur Vertiefung: L. Ramalho, *Fluent Python*, Kapitel 6.

2.6 enumerate, zip und Comprehensions

enumerate. Aus Teil 1 kennen Sie `enumerate`: Es liefert zu jedem Element seine Nummer. Sind die Elemente selbst Tupel, braucht das Auspacken eine zusätzliche Klammer.

```
hits = [("Gehirn", 0.89), ("Zeit", 0.72), ("Koran", 0.65)]
❶ for rank, (title, score) in enumerate(hits, start=1):
    print(rank, title, score)
```

❶ `start=1` lässt die Zählung bei 1 beginnen

```
1 Gehirn 0.89
2 Zeit 0.72
3 Koran 0.65
```

Ohne die Klammer, also mit `for rank, title, score in enumerate(hits)`, bricht die Schleife mit `ValueError: not enough values to unpack (expected 3, got 2)` ab. `enumerate` liefert je Durchlauf zwei Werte: die Nummer und den Treffer.

zip. `zip` durchläuft mehrere Listen gleichzeitig und liefert je Durchlauf ein Tupel aus den Elementen an derselben Position.

```
titles = ["Gehirn", "Zeit", "Koran"]
scores = [0.89, 0.72, 0.65]
❶ for title, score in zip(titles, scores):
    print(title, score)
❷ print(list(zip(titles, scores)))
❸ print(list(zip(titles, [0.89, 0.72])))
```

- ❶ `zip` liefert je Durchlauf ein Paar aus beiden Listen
- ❷ `list(...)` macht aus dem Ergebnis von `zip` eine Liste von Tupeln
- ❸ bei ungleich vielen Elementen hört `zip` beim Ende der kürzeren Liste auf, ohne Fehlermeldung

```
Gehirn 0.89
Zeit 0.72
Koran 0.65
[('Gehirn', 0.89), ('Zeit', 0.72), ('Koran', 0.65)]
[('Gehirn', 0.89), ('Zeit', 0.72)]
```

Das stille Abschneiden ist eine Fehlerquelle. Gehören zwei Listen zusammen, etwa Absätze und ihre Vektoren, und eine ist versehentlich kürzer, fehlen im Ergebnis Elemente, ohne dass etwas auffällt. Wenn beide Listen gleich viele Elemente haben müssen, schreiben Sie `zip(titles, scores, strict=True)`. Dann bricht `zip` bei ungleich vielen Elementen mit einem `ValueError` ab.

List Comprehensions. Viele Schleifen bauen aus einer Liste eine neue Liste. Für dieses Muster hat Python eine Kurzform, die List Comprehension. Die nächste Zelle zeigt denselben Rechenweg zweimal.

```

hits = [("Gehirn", 0.89), ("Zeit", 0.72), ("Koran", 0.65)]

titles = []
for title, score in hits:
    titles.append(title)          # die Schleife: je Treffer den Titel anhängen
print(titles)
❶ titles = [title for title, score in hits]
print(titles)
❷ good = [title for title, score in hits if score >= 0.7]
print(good)
❸ print([title.upper() for title, score in hits])

```

- ❶ die Comprehension steht in eckigen Klammern: vorn der Ausdruck für ein Element des Ergebnisses, danach die Schleife
- ❷ eine Bedingung am Ende übernimmt nur die Elemente, für die sie gilt
- ❸ der Ausdruck vorn darf rechnen oder Methoden aufrufen

```

['Gehirn', 'Zeit', 'Koran']
['Gehirn', 'Zeit', 'Koran']
['Gehirn', 'Zeit']
['GEHIRN', 'ZEIT', 'KORAN']

```

Schleife oder Comprehension? Beide Fassungen leisten dasselbe. Die Comprehension passt, wenn aus einer Liste in einer gut lesbaren Zeile eine neue Liste wird: ein Ausdruck, eine Schleife, höchstens eine Bedingung. Sobald mehrere Schritte, Zwischenergebnisse oder verschachtelte Schleifen nötig sind, ist die gewöhnliche Schleife leichter zu lesen und zu prüfen. Eine Comprehension, die Sie beim zweiten Lesen nicht sofort verstehen, ist zu lang.

Dieselbe Schreibweise gibt es für Dicts.

```

❶ id_to_title = {p["id"]: p["title"] for p in sample}
print(id_to_title)

```

- ❶ geschweifte Klammern mit Schlüssel: Wert vorn bilden ein Dict: hier von der Nummer zum Titel

```
{6: 'Mexiko-Stadt', 108: 'San Diego', 78: 'Mexiko-Stadt'}
```

Mehrere for und if. Eine Comprehension darf mehr als ein `for` und mehr als ein `if` enthalten. Gelesen wird sie von links nach rechts wie verschachtelte Schleifen: Das erste `for` ist die äußere Schleife, das zweite die innere. Ein `if` darf die Namen verwenden, die links davon eingeführt sind, und zwei `if` hintereinander wirken wie eine Bedingung mit `and`. Die nächste Zeile zeigt eine solche Comprehension und darunter dieselbe Rechnung als Schleife.

```

rows = [[1, 12, 5], [20, 3]]

picked = [x for row in rows for x in row if x > 2 if x < 20]
print(picked)

picked = []
for row in rows:          # das erste for: die äußere Schleife
    for x in row:          # das zweite for: die innere Schleife
        if x > 2 and x < 20: # zwei if hintereinander wirken wie and
            picked.append(x)
print(picked)

```

```

[12, 5, 3]
[12, 5, 3]

```

Eine solche Zeile ist schnell geschrieben und langsam gelesen. Sie ist ein Fall für die Regel oben: Mit verschachtelten Schleifen und mehreren Bedingungen ist die gewöhnliche Schleife leichter zu prüfen.

 Aufgabe 2.6 #1

10 min

Schreiben Sie beide Funktionen als List Comprehension.

1. `titles_of(hits)` gibt die Titel einer Trefferliste zurück, in derselben Reihenfolge.
2. `hits_from(hits, limit)` gibt die Treffer zurück, deren Score mindestens `limit` beträgt. Das Ergebnis enthält die vollständigen Paare.

Eine Trefferliste `hits` ist eine Liste von Paaren aus Titel und Score, zum Beispiel `[("Gehirn", 0.89), ("Zeit", 0.72)]`.

Zusatzaufgabe: Schreiben Sie `hits_from` in der Zelle nach der Prü fzelle zusätzlich als gewöhnliche Schleife unter dem Namen `hits_from_loop` und vergleichen Sie beide Fassungen mit `==` an einem Beispiel. Welche Fassung lesen Sie leichter?

Hinweis: In `hits_from` ist der Ausdruck vorn das ganze Paar. Entweder packen Sie in der Schleife aus und setzen das Paar als `(title, score)` wieder zusammen, oder Sie laufen mit `for hit in hits` über die Paare und prüfen `hit[1]`.

[→ Lösung im Anhang](#)

```
def titles_of(hits):
    # TODO: Ihre Lösung hier
    ...

def hits_from(hits, limit):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prü fzelle – einfach ausführen
_hits = [("Gehirn", 0.89), ("Zeit", 0.72), ("Koran", 0.65)]
assert titles_of(_hits) is not None, "titles_of gibt noch nichts zurück (return vergessen?)"
assert titles_of(_hits) == ["Gehirn", "Zeit", "Koran"], "die Titel in der Reihenfolge der Treffer"
assert titles_of([]) == []
assert hits_from(_hits, 0.7) is not None, "hits_from gibt noch nichts zurück (return vergessen?)"
_expected = [("Gehirn", 0.89), ("Zeit", 0.72)]
assert hits_from(_hits, 0.7) == _expected, "die vollständigen Paare ab der Grenze"
assert hits_from(_hits, 0.72) == _expected, "ein Score genau auf der Grenze gehört dazu"
assert hits_from(_hits, 0.95) == [], "kein Treffer erreicht die Grenze"
assert _hits == [("Gehirn", 0.89), ("Zeit", 0.72), ("Koran", 0.65)], "die Eingabe bleibt unverändert"
print("Alle Prüfungen bestanden ✓")
```

```
# Zusatzaufgabe: hits_from als gewöhnliche Schleife, danach beide Fassungen vergleichen
# TODO: Ihre Lösung hier
...
```

 Aufgabe 2.6 #2

10 min

Review. Die Funktion `long_words` in der Zelle darunter ist als verschachtelte Comprehension geschrieben. Sie verwendet Ihre Funktion `clean_words` aus dem Kapitel [2.4](#) und liefert zu einer Liste von Absätzen alle bereinigten Wörter mit mehr als zwölf Zeichen, die keinen Bindestrich enthalten.

1. Führen Sie die Zelle aus und wenden Sie die Funktion in einer neuen Zelle auf `sample` an.
2. Schreiben Sie `long_words_loop(records)` als gewöhnliche Schleife mit demselben Ergebnis.
3. Notieren Sie als Kommentar, welche Fassung Sie in einem Review schneller prüfen könnten, und warum.

Hinweis: Lesen Sie die Zeile von links nach rechts wie im Beispiel vor dieser Aufgabe. `for p in records` ist die äußere Schleife über die Absätze, `for w in clean_words(p["text"])` die innere über die Wörter eines Absatzes. Beide `if` prüfen das Wort `w` und wirken zusammen wie eine Bedingung mit `and`.

[→ Lösung im Anhang](#)

```
# Die vorgelegte Fassung
def long_words(records):
    return [w for p in records for w in clean_words(p["text"]) if len(w) > 12 if "-" not in w]

def long_words_loop(records):
    # TODO: Ihre Lösung hier
    ...

# Welche Fassung könnten Sie schneller prüfen, und warum?
# TODO: Ihre Lösung hier
...
```

```
# Prüfwelle – einfach ausführen
_long = long_words_loop(sample)
assert _long is not None, "long_words_loop gibt noch nichts zurück (return vergessen?)"
assert _long == long_words(sample), "beide Fassungen liefern dasselbe, in derselben Reihenfolge"
assert long_words_loop([]) == []
assert long_words_loop(passages[:20]) == long_words(passages[:20]), "auch auf 20 Absätzen"
print("Alle Prüfungen bestanden ✓")
```

Lernfragen 2.6 #1

1. Was gibt `[len(w) for w in ["gpu", "tensor"] if len(w) > 3]` zurück?
2. Zwei Listen `passages` und `vectors` gehören elementweise zusammen. Durch einen Fehler beim Laden hat `vectors` nur 399 Einträge. Was passiert bei `for p, v in zip(passages, vectors):`, und wie fällt der Fehler auf?

[→ Lösung im Anhang](#)

Zum Nachschlagen: [Python-Tutorial 5.1.3, List Comprehensions](#) und [5.6, Schleifentechniken](#).

2.7 Sortieren und Funktionen als Argumente

Im Termin haben Sie die besten Treffer mit einer eigenen Schleife gesucht. Python bringt dafür `sorted` mit. Dieses Kapitel zeigt, wie Sie damit nach einem selbst gewählten Kriterium ordnen.

```
scores = [0.72, 0.89, 0.65]
① print(sorted(scores))
② print(sorted(scores, reverse=True))
print(scores)                                # die ursprüngliche Liste ist unverändert
```

- ① `sorted` gibt eine neue, aufsteigend geordnete Liste zurück
- ② `reverse=True` ordnet absteigend

```
[0.65, 0.72, 0.89]
[0.89, 0.72, 0.65]
[0.72, 0.89, 0.65]
```

Kurz überlegen

Eine Trefferliste besteht aus Paaren `(title, score)`. Wonach ordnet `sorted(hits)`, wenn Sie nichts weiter angeben?

```
hits = [("Zeit", 0.72), ("Gehirn", 0.89), ("Koran", 0.65)]
print(sorted(hits))
```

Das Kriterium angeben: `key`. Wonach geordnet wird, legt das Argument `key` fest. Es erwartet eine Funktion. `sorted` ruft sie für jedes Element einmal auf und ordnet nach den Ergebnissen. Die Elemente selbst bleiben unverändert.

```
words = ["Tensor", "gpu", "Python", "CPU"]
① print(sorted(words))
② print(sorted(words, key=len))
③ print(sorted(words, key=str.lower))
```

- ① ohne `key` vergleicht Python die Zahlenwerte der Zeichen: Die Buchstaben A bis Z stehen vor a bis z
- ② `key=len` ordnet nach der Anzahl der Zeichen: übergeben wird die Funktion `len` selbst, ohne Klammern
- ③ `key=str.lower` ordnet nach der kleingeschriebenen Form: `str.lower` ist die Methode `lower` als Funktion

```
['CPU', 'Python', 'Tensor', 'gpu']
['gpu', 'CPU', 'Tensor', 'Python']
['CPU', 'gpu', 'Python', 'Tensor']
```

Funktionen sind Werte

`key=len` übergibt die Funktion `len`. `sorted` ruft sie später selbst auf. `key=len()` würde `len` sofort aufrufen und das Ergebnis übergeben. Dieser Aufruf scheitert schon vorher, weil `len` ohne Argument aufgerufen wird: `TypeError: len() takes exactly one argument (0 given)`. Eine Funktion ohne Klammern ist ein Wert wie eine Zahl oder ein String: Sie lässt sich übergeben, in einer Variablen ablegen und später aufrufen. Dasselbe haben Sie in der ersten Lerneinheit bei `measure` gesehen, das eine Funktion als Argument bekommt.

Bei deutschen Texten ist die Reihenfolge ohne `key` oft nicht die erwartete: `sorted(["Zebra", "apfel", "Äpfel"])` liefert `['Zebra', 'apfel', 'Äpfel']`. Python vergleicht Strings Zeichen für Zeichen nach den Unicode-Zahlenwerten. Dabei stehen A bis Z vor a bis z, und die deutschen Umlaute stehen hinter diesen Buchstaben. `key=str.lower` hebt den Unterschied zwischen Groß- und Kleinbuchstaben auf, die Umlaute bleiben am Ende. Für eine Sortierung nach deutschen Regeln gibt es das Modul `locale`.

Eigene Schlüsselfunktionen. Für eine Trefferliste gibt es keine fertige Funktion, die den Score liefert. Sie schreiben sie selbst.

```
hits = [("Zeit", 0.72), ("Gehirn", 0.89), ("Koran", 0.65)]

def score_of(hit):
    return hit[1] # der Score steht an Position 1 des Paares
① print(sorted(hits, key=score_of, reverse=True))
② print(sorted(hits, key=lambda hit: hit[1], reverse=True))
```

- ① die eigene Funktion wird wie `len` ohne Klammern übergeben
- ② `lambda hit: hit[1]` ist eine Funktion ohne Namen: vor dem Doppelpunkt steht der Parameter, dahinter der Ausdruck, dessen Wert sie zurückgibt

```
[('Gehirn', 0.89), ('Zeit', 0.72), ('Koran', 0.65)]
[('Gehirn', 0.89), ('Zeit', 0.72), ('Koran', 0.65)]
```

Benannte Funktion oder `lambda`? Beide Zeilen leisten dasselbe. Ein `lambda` eignet sich für einen kurzen Ausdruck, der nur an dieser einen Stelle gebraucht wird. Es kann nur einen einzigen Ausdruck enthalten. Sobald die Funktion mehr tun soll, an mehreren Stellen gebraucht wird oder ein Name das Lesen erleichtert, ist `def` die bessere Wahl.

`sorted` oder `sort`? Neben der Funktion `sorted` gibt es die Listen-Methode `sort`. Sie ordnet die Liste selbst um und gibt `None` zurück.

```
scores = [0.72, 0.89, 0.65]
result = scores.sort() # ordnet scores selbst um
print(scores, result)

[0.65, 0.72, 0.89] None
```

Die Zeile `scores = scores.sort()` ist die `None`-Falle aus Teil 1 in neuer Form: Danach ist `scores` keine Liste mehr. Wichtiger für die Arbeit mit Daten ist der andere Unterschied. `sort` verändert die Liste, auf der es aufgerufen wird. Ist das die Liste, die eine Funktion als Argument bekommen hat, verändert die Funktion die Daten des Aufrufers.

Gleichstände. Haben zwei Elemente denselben Schlüssel, behalten sie die Reihenfolge, die sie in der Eingabe hatten. Das gilt auch mit `reverse=True`. Diese Eigenschaft heißt Stabilität.

```
hits = [("Koran", 0.80), ("Zeit", 0.74), ("Gehirn", 0.80)]
print(sorted(hits, key=lambda hit: hit[1], reverse=True))    # Koran bleibt vor Gehirn

[('Koran', 0.8), ('Gehirn', 0.8), ('Zeit', 0.74)]
```

Ein Dict nach seinen Werten ordnen. `items()` liefert Paare aus Schlüssel und Wert. Mit dem Wert als Schlüssel für `sorted` entsteht eine Rangliste.

```
counts = {"gpu": 2, "die": 5, "und": 3}
print(sorted(counts.items(), key=lambda item: item[1], reverse=True))

[('die', 5), ('und', 3), ('gpu', 2)]
```

Dieselbe Zeile bestimmt die häufigsten Wörter des Korpus. Die nächste Zelle verwendet dafür `word_counts` aus dem Kapitel 2.4. Wenn Sie dieses Kapitel übersprungen haben, gibt es `word_counts` nicht, und die Zelle bricht mit einem `NameError` ab. Die Ausgabe steht dann im Skript.

```
top = sorted(word_counts.items(), key=lambda item: item[1], reverse=True)[:8]
for word, n in top:
    print(f"{n:5} {word}")

2337  der
2171  die
1592  und
1191  in
823   von
698   den
647   im
616   das
```

Aufgabe 2.7 #1

10 min

Schreiben Sie die Funktion `top_k(hits, k)`. Sie gibt die `k` Treffer mit dem höchsten Score zurück, absteigend nach dem Score geordnet. Für `k` dürfen Sie eine ganze Zahl ab 0 voraussetzen. Sind weniger als `k` Treffer vorhanden, gibt die Funktion alle vorhandenen zurück, mit `k = 0` ist das Ergebnis leer. Bei gleichem Score bleibt die Reihenfolge der Eingabe erhalten. Die Eingabeliste bleibt unverändert. Im Termin hat diese Aufgabe eine Schleife mit Arbeitskopie gebraucht.

Zusatzaufgabe: Im Kapitel 2.3 hat die Prü fzelle zu `group_by_title` das Dict `groups` angelegt. Es ordnet jedem Titel die Liste seiner Absatznummern zu. Voraussetzung ist, dass Sie die Aufgabe dort gelöst haben. Geben Sie in der Zelle nach der Prü fzelle die fünf Titel mit den meisten Absätzen aus. Bei gleicher Anzahl soll der Titel alphabetisch entscheiden. Als Schlüssel eignet sich dafür ein Tupel aus der negativen Anzahl und dem Titel.

Hinweis: `sorted` mit `key` und `reverse=True` liefert die geordnete Liste, ein Slice schneidet die ersten `k` heraus.

[→ Lösung im Anhang](#)

```
def top_k(hits, k):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfwelle – einfach ausführen
_hits = [("Bern", 0.61), ("Koran", 0.80), ("Zeit", 0.74), ("Gehirn", 0.80)]
_before = list(_hits) # Kopie zum Vergleich nach den Aufrufen
_top = top_k(_hits, 2)
assert _top is not None, "top_k gibt noch nichts zurück (return vergessen?)"
assert _top == [("Koran", 0.80), ("Gehirn", 0.80)], "Gleichstand in der Reihenfolge der Eingabe"
assert top_k(_hits, 3)[2] == ("Zeit", 0.74), "absteigend nach dem Score"
assert len(top_k(_hits, 10)) == 4, "k größer als die Liste: alle Treffer"
assert top_k(_hits, 10)[3] == ("Bern", 0.61), "der schwächste Treffer steht am Ende"
assert top_k(_hits, 0) == []
assert _hits == _before, "die Eingabeliste bleibt unverändert"
print("Alle Prüfungen bestanden ✓")
```

```
# Zusatzaufgabe: die fünf Titel mit den meisten Absätzen
# TODO: Ihre Lösung hier
...
```

Aufgabe 2.7 #2

10 min

Review. Die Fassung `top_k_v2` in der Zelle darunter soll dieselben Anforderungen erfüllen wie `top_k`: die `k` besten Treffer, absteigend geordnet, Gleichstände in der Reihenfolge der Eingabe, Eingabeliste unverändert.

1. Führen Sie die Zelle aus und probieren Sie die Funktion in einer neuen Zelle aus.
2. Schreiben Sie in die zweite Zelle einen eigenen Test, der zeigt, ob die Funktion alle Anforderungen erfüllt.
3. Notieren Sie Ihr Urteil als Kommentar und korrigieren Sie die Funktion, falls nötig, direkt in der Zelle.

Hinweis: Die Checkliste „Wie prüfe ich eine Lösung?“ aus Teil 1 nennt vier Fragen: Normalfall, Grenzfall, Rückweg, Nebenwirkungen. Gehen Sie alle vier durch und schreiben Sie zu jeder mindestens ein `assert`. Die Lösung nennt hinter jedem `assert`, welche Frage es prüft.

[→ Lösung im Anhang](#)

```
# Die zu prüfende Fassung – korrigieren Sie nur, was wirklich falsch ist
def top_k_v2(hits, k):
    hits.sort(key=lambda hit: hit[1], reverse=True)
    return hits[:k]
```

```
# Ihr Test:
# TODO: Ihre Lösung hier
...
```

```
# Prüfwelle – einfach ausführen
_hits = [("Bern", 0.61), ("Koran", 0.80), ("Zeit", 0.74), ("Gehirn", 0.80)]
_before = list(_hits) # Kopie zum Vergleich nach den Aufrufen
assert top_k_v2(_hits, 2) == [("Koran", 0.80), ("Gehirn", 0.80)], "die zwei besten Treffer"
assert _hits == _before, "die Eingabeliste bleibt unverändert: sorted statt sort"
assert top_k_v2(_hits, 0) == []
print("Alle Prüfungen bestanden ✓")
```

Überblick: Eingebaute Funktionen

Die eingebauten Funktionen dieses Teils und einige weitere zum Nachschlagen. Dabei steht `xs` für eine Liste, ein Tupel oder einen anderen Container. Keine der Funktionen verändert ihre Eingabe.

Werkzeug	Tut was	Gibt zurück
<code>len(xs)</code>	Anzahl der Elemente einer Liste, eines Tupels, Strings, Dicts oder Sets	<code>int</code>
<code>sorted(xs, key=f, reverse=True)</code>	ordnet nach dem Schlüssel <code>f</code>	neue Liste
<code>reversed(xs)</code>	durchläuft eine Liste, ein Tupel oder einen String rückwärts	etwas zum Durchlaufen, mit <code>list(...)</code> eine Liste
<code>min(xs), max(xs)</code>	kleinstes und größtes Element, auch mit <code>key=f</code>	ein Element
<code>sum(xs)</code>	Summe, die Elemente sind Zahlen	eine Zahl
<code>any(xs)</code>	prüft, ob mindestens ein Element wahr ist	<code>bool</code>
<code>all(xs)</code>	prüft, ob alle Elemente wahr sind	<code>bool</code>
<code>enumerate(xs, start=1)</code>	Paare aus Nummer und Element	etwas zum Durchlaufen
<code>zip(xs, ys)</code>	Paare aus Elementen an derselben Position	etwas zum Durchlaufen

`sorted`, `min`, `max`, `any`, `all`, `enumerate` und `zip` arbeiten auf allem, was sich mit `for` durchlaufen lässt. `len` braucht einen Container, dessen Anzahl feststeht, und `reversed` eine Liste, ein Tupel oder einen String. `len(zip(xs, ys))` und `reversed` auf einem Set brechen deshalb mit einem `TypeError` ab.

Zwei Beispiele: `max(hits, key=lambda hit: hit[1])` liefert den Treffer mit dem höchsten Score. `any([score > 0.8 for title, score in hits])` prüft, ob mindestens ein Score über 0,8 liegt.

Lernfragen 2.7 #2

1. Welche Zeilen ordnen `words` nach der Anzahl der Zeichen und liefern eine Liste?

- `sorted(words, key=len)`
- `sorted(words, key=len())`
- `words.sort(key=len)`
- `sorted(words, key=lambda w: len(w))`

2. `sorted(hits, key=lambda hit: hit[1], reverse=True)[-1]` und `sorted(hits, key=lambda hit: hit[1])` ordnen beide aufsteigend nach dem Score. Liefern sie immer dasselbe Ergebnis?

→ [Lösung im Anhang](#)

Zum Nachschlagen: die Anleitung [Sortieren](#) in der Python-Dokumentation und [Python-Tutorial 4.9.6. Lambda-Ausdrücke](#).

2.8 Abschluss

Zum Abschluss setzen Sie die Werkzeuge zusammen und beantworten die Leitfrage vom Anfang: Wie schreiben Sie eine Auswahl aus einer Trefferliste, und wie zeigen Sie, dass sie jede Anforderung erfüllt? Die erste Aufgabe schreiben Sie selbst, in der zweiten beurteilen Sie eine vorgelegte Lösung. Beide arbeiten mit der kleinen Trefferliste aus der Vorgabe. Sie enthält zwei Treffer mit gleichem Score und einen Treffer, dessen Score genau 0,5 beträgt.

```
# Vorgabe: eine kleine Trefferliste zum Entwickeln und Testen – einfach ausführen
demo_hits = [("Bern", 0.61), ("Koran", 0.80), ("Zeit", 0.74), ("Gehirn", 0.80),
              ("Iran", 0.50), ("Sichuan", 0.42)]
```

 Aufgabe 2.8 #1

15 min

Schreiben Sie die Funktion `select_hits(hits, threshold, k)`. Sie wählt aus einer Trefferliste aus und erfüllt vier Anforderungen:

1. Übernommen werden die Treffer, deren Score mindestens `threshold` beträgt.
2. Das Ergebnis ist absteigend nach dem Score geordnet. Bei gleichem Score bleibt die Reihenfolge der Eingabe erhalten.
3. Zurückgegeben werden höchstens `k` Treffer. `k` ist eine ganze Zahl ab 0, mit `k = 0` ist das Ergebnis eine leere Liste.
4. Die Eingabeliste bleibt unverändert.

Schreiben Sie in die zweite Zelle zu jeder Anforderung mindestens einen eigenen Test mit `assert` und notieren Sie als Kommentar dahinter, welche Anforderung der Test prüft. Führen Sie erst danach die Prü fzelle aus.

Hinweis: Die Aufgabe besteht aus drei Schritten, für die Sie je ein Werkzeug kennen: auswählen (Comprehension mit Bedingung oder `hits_from`), ordnen (`sorted` mit `key`), kürzen (`Slice`). Die Tests schreiben Sie am leichtesten mit `demo_hits` und Werten, deren Ergebnis Sie von Hand bestimmen können.

[→ Lösung im Anhang](#)

```
def select_hits(hits, threshold, k):
    # TODO: Ihre Lösung hier
    ...
```

```
# Ihre Tests: je Anforderung mindestens ein assert, dahinter als Kommentar die geprüfte Anforderung
# TODO: Ihre Lösung hier
...
```

```
# Prü fzelle – einfach ausführen
_hits = [("Bern", 0.61), ("Koran", 0.80), ("Zeit", 0.74), ("Gehirn", 0.80),
        ("Iran", 0.50), ("Sichuan", 0.42)]
_before = list(_hits) # Kopie zum Vergleich nach den Aufrufen
_expected = [("Koran", 0.80), ("Gehirn", 0.80), ("Zeit", 0.74), ("Bern", 0.61), ("Iran", 0.50)]
_res = select_hits(_hits, 0.5, 10)
assert _res is not None, "select_hits gibt noch nichts zurück (return vergessen?)"
assert _res == _expected, "ab der Grenze 0.5, absteigend, Gleichstand in der Reihenfolge der Eingabe"
assert select_hits(_hits, 0.5, 3) == _expected[:3], "höchstens k Treffer"
assert select_hits(_hits, 0.5, 0) == [], "k = 0 ergibt eine leere Liste"
assert select_hits(_hits, 0.9, 3) == [], "kein Treffer erreicht die Grenze"
assert select_hits([], 0.5, 3) == [], "eine leere Trefferliste"
assert _hits == _before, "die Eingabeliste bleibt unverändert"
print("Alle Prüfungen bestanden ✓")
```

 Aufgabe 2.8 #2

10 min

Review mit offenem Ausgang. Die Fassung `select_hits_v2` in der Zelle darunter soll dieselben vier Anforderungen erfüllen. Entscheiden Sie wie in einem echten Review:

1. Lesen Sie die Funktion und führen Sie die Zelle aus.
2. Prüfen Sie jede der vier Anforderungen mit einem eigenen Test in der zweiten Zelle. Ihre Tests aus der vorigen Aufgabe können Sie anpassen.
3. Notieren Sie Ihr Urteil als Kommentar und begründen Sie es am Code: Warum erfüllt die Funktion jede Anforderung, oder an welcher Stelle verletzt sie eine? Korrigieren Sie nur, was wirklich falsch ist.

Führen Sie die Zelle auf jeden Fall aus, auch wenn Sie nichts korrigieren.

[→ Lösung im Anhang](#)

```
# Die zu prüfende Fassung – korrigieren Sie nur, was wirklich falsch ist
def select_hits_v2(hits, threshold, k):
    result = sorted(hits, key=lambda hit: -hit[1])
    while len(result) > 0 and result[-1][1] < threshold:
        result.pop()
    return result[:k]
```

```
# Ihre Tests und Ihr Urteil als Kommentar:
# TODO: Ihre Lösung hier
...
```

```
# Prüfwelle – einfach ausführen
_hits = [("Bern", 0.61), ("Koran", 0.80), ("Zeit", 0.74), ("Gehirn", 0.80),
         ("Iran", 0.50), ("Sichuan", 0.42)]
_before = list(_hits) # Kopie zum Vergleich nach den Aufrufen
_expected = [("Koran", 0.80), ("Gehirn", 0.80), ("Zeit", 0.74), ("Bern", 0.61), ("Iran", 0.50)]
assert select_hits_v2(_hits, 0.5, 10) == _expected, "die Funktion muss alle vier Anforderungen erfüllen"
assert select_hits_v2(_hits, 0.5, 0) == []
assert select_hits_v2(_hits, 0.9, 3) == []
assert select_hits_v2([], 0.5, 3) == []
assert _hits == _before, "die Eingabeliste bleibt unverändert"
print("Alle Prüfungen bestanden ✓")
```

Ein Review kann mit dem Ergebnis enden, dass nichts zu ändern ist. Tests allein belegen das nicht: Sie zeigen, dass die geprüften Fälle stimmen, über alle anderen Eingaben sagen sie nichts. Für die Freigabe braucht es deshalb beides. Die Tests decken jede Anforderung ab, auch an den Grenzen. Und am Code lässt sich begründen, warum jede Anforderung für alle zulässigen Eingaben erfüllt ist. Ein ungewohnter Stil ist für sich kein Fehler. Eine Änderung braucht einen benennbaren Grund, zum Beispiel eine verletzte Anforderung oder schwer lesbaren Code, denn jede Änderung an geprüftem Code kann neue Fehler einführen.

Aufgabe 2.8 #3

freiwillig, 10 min

Zusatzaufgabe zu Klassen. Im Termin haben Sie die Klasse `Corpus` um Methoden ergänzt. Diese Aufgabe wiederholt die Schreibweise an einem kleineren Beispiel. Schreiben Sie die Klasse `HitList`:

1. `__init__(self, hits)` legt die Trefferliste im Attribut `hits` ab.
2. Die Methode `select(self, threshold, k)` gibt die Auswahl nach den vier Anforderungen zurück. Sie darf `select_hits` aufrufen.
3. Die Sondermethode `__len__(self)` gibt die Anzahl der Treffer zurück.

Hinweis: Jede Methode erreicht die Trefferliste des Objekts über `self.hits`. Der Aufruf lautet später `HitList(demo_hits).select(0.5, 3)`.

[→ Lösung im Anhang](#)

```
class HitList:
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfwelle – einfach ausführen (legt zwei Objekte mit eigenen Daten an)
_a = HitList([("Bern", 0.61), ("Koran", 0.80), ("Iran", 0.50)])
_b = HitList([("Zeit", 0.74)])
assert len(_a) == 3 and len(_b) == 1, "__len__ zählt die Treffer des Objekts (self.hits)"
assert _a.select(0.5, 2) == [("Koran", 0.80), ("Bern", 0.61)], "select nutzt die Treffer des Objekts"
assert _b.select(0.5, 2) == [("Zeit", 0.74)], "ein zweites Objekt hat seine eigenen Treffer"
assert _a.select(0.9, 2) == []
print("Alle Prüfungen bestanden ✓")
```

Überblick: Welches Werkzeug passt zu welcher Datenoperation?

Die Werkzeuge dieses Teils, geordnet nach dem, was Sie mit den Daten vorhaben.

Ich will ...	Werkzeug	Beispiel
einen Wert über einen Namen nachschlagen	Dict	<code>p["title"], p.get("author", "unbekannt")</code>
Datensätze über eine Nummer finden	Dict von der Nummer zum Datensatz	<code>by_id[108]</code>
zählen, wie oft etwas vorkommt	Dict mit <code>get</code>	<code>counts[w] = counts.get(w, 0) + 1</code>
Datensätze gruppieren	Dict aus Listen	<code>groups[title].append(p["id"])</code>
Duplikate entfernen	Set	<code>set(words)</code>
prüfen, ob etwas vorkommt	<code>in</code> , bei vielen Prüfungen auf einem Set	<code>"gpu" in vocabulary</code>
gemeinsame Werte finden	Schnittmenge	<code>a & b</code>
Text in Wörter zerlegen und zusammensetzen	<code>split</code> und <code>join</code>	<code>text.lower().split()</code>
einen Ausschnitt nehmen	Slice	<code>hits[:k], xs[::-1]</code>
jedes Element umformen	Comprehension oder Schleife	<code>[title for title, score in hits]</code>
nach einer Bedingung auswählen	Comprehension mit <code>if</code> oder Schleife	<code>[h for h in hits if h[1] >= 0.5]</code>
Position und Element zusammen durchlaufen	<code>enumerate</code>	<code>for rank, hit in enumerate(hits, start=1):</code>
zwei Listen paarweise durchlaufen	<code>zip</code>	<code>for p, v in zip(passages, vectors):</code>
ordnen	<code>sorted</code> mit <code>key</code>	<code>sorted(hits, key=lambda h: h[1], reverse=True)</code>
unabhängige Datensätze erzeugen	jedes Dict kopieren, bei Dicts mit Zahlen und Strings als Werten	<code>[r.copy() for r in records]</code>

Vertiefung: Was Python außerdem mitbringt

NICHT PRÜFUNGSRELEVANT

Für mehrere Muster dieses Teils bringt Python fertige Werkzeuge mit. Für diesen Kurs brauchen Sie sie nicht. In fremdem Code begegnen sie Ihnen aber, deshalb zeigt diese Box je ein Beispiel.

Zählen. Counter aus dem Modul collections zählt die Elemente einer Liste und liefert die häufigsten:

```
from collections import Counter
counts = Counter(["die", "gpu", "die"])
counts["die"]           # 2
counts.most_common(1)   # [('die', 2)]
```

Gruppieren. Ein defaultdict legt einen fehlenden Eintrag beim ersten Zugriff selbst an. Die Dict-Methode setdefault leistet dasselbe für ein gewöhnliches Dict:

```
from collections import defaultdict
by_title = defaultdict(list)           # fehlender Schlüssel: leere Liste
by_title["Bern"].append(11)            # kein KeyError

by_title = {}
by_title.setdefault("Bern", []).append(11) # legt die Liste an, falls der Schlüssel fehlt
```

Ein Dict aus zwei Listen. zip liefert die Paare, dict macht daraus Einträge:

```
dict(zip(["Gehirn", "Zeit"], [0.89, 0.72])) # {'Gehirn': 0.89, 'Zeit': 0.72}
```

Auspacken mit Stern. Ein Name mit Stern nimmt alle übrigen Werte als Liste auf:

```
first, *rest = [1, 2, 3, 4]           # first ist 1, rest ist [2, 3, 4]
```

Zuweisung an einen Slice. Sie ersetzt den Ausschnitt in der vorhandenen Liste:

```
xs = [0, 1, 2, 3]
xs[1:3] = [9]                         # xs ist jetzt [0, 9, 3]
```

frozenset. Ein unveränderliches Set. Es ist hashbar und darf deshalb Schlüssel eines Dicts oder Element eines Sets sein:

```
similarity = {frozenset(["gpu", "cpu"]): 0.4}
similarity[frozenset(["cpu", "gpu"])]      # 0.4, die Reihenfolge spielt keine Rolle
```

Generatoren. Eine Funktion mit yield liefert ihre Werte einzeln und erst dann, wenn sie abgerufen werden:

```
def squares(n):
    for i in range(n):
        yield i * i

list(squares(4))                       # [0, 1, 4, 9]
```

itertools. Das Modul enthält Bausteine für Schleifen, zum Beispiel alle Paare aus einer Liste:

```
from itertools import combinations
list(combinations(["a", "b", "c"], 2))   # [('a', 'b'), ('a', 'c'), ('b', 'c')]
```

map und filter. Beide leisten dasselbe wie eine Comprehension:

```
words = ["gpu", "tensor"]
list(map(len, words))                  # [3, 6]
[len(w) for w in words]                # dasselbe als Comprehension
list(filter(lambda w: len(w) > 3, words)) # ['tensor']
[w for w in words if len(w) > 3]       # dasselbe als Comprehension
```

→ Nächste Schritte

- In der nächsten Lerneinheit übernimmt NumPy. Slicing, Tupel und das Denken in „jedes Element umformen“ brauchen Sie dort wieder.
- Teil 3 des Grundkurses folgt mit der nächsten Lerneinheit. Er beginnt mit Dateien und Fehlerbehandlung und vertieft Funktionen und Klassen.
- Zum Nachschlagen neben dem Kurs: das [offizielle Python-Tutorial](#), Kapitel 5.

Quellen und Referenzen

- Python Software Foundation: [Python-Tutorial](#), Kapitel 5 (Datenstrukturen), die Anleitung [Sortieren](#) und die Beschreibung der [eingebauten Typen](#). Die Kapitel dieses Teils nennen ihre Abschnitte direkt.
- Python Software Foundation: [Status of Python versions](#) (Pflegezeiträume der Python-Versionen).
- L. Ramalho: *Fluent Python*, 2. Auflage, Kapitel 2, 3 und 6. Vertiefung zu Listen und Tupeln, Dicts, Sets und Verweisen.
- W. McKinney: *Python for Data Analysis*, 3. Auflage, Kapitel 3. Die eingebauten Datenstrukturen aus Sicht der Datenanalyse.
- Datengrundlage: [GermanQuAD](#) (deepset), Lizenz CC BY 4.0. Die Absatztexte stammen aus der deutschen Wikipedia (CC BY-SA). Attribution auch in `data/appetizer_passages.json`.

Dieses Material wurde mit Unterstützung von KI-Werkzeugen entwickelt (Text und Code: Claude und ChatGPT). Alle Inhalte sind von Prof. Dr. Stephan Kurpuweit konzipiert, geprüft und verantwortet.

Anhang: Lösungen

2.3 Tupel, Dicts und verschachtelte Daten

Aufgabe 2.3 #1

2 min

Die Trefferliste in der Zelle darunter enthält je Treffer drei Werte: Titel, Score und Absatznummer. Die Schleife soll für jeden Treffer eine Zeile wie `0.89 Gehirn` in der Liste `lines` sammeln. Sie bricht stattdessen mit einem `ValueError` ab. Führen Sie die Zelle aus, lesen Sie die letzte Zeile der Fehlermeldung und korrigieren Sie den Fehler direkt in der Zelle.

```
hits = [("Gehirn", 0.89, 92), ("Zeit", 0.72, 157)]    # Titel, Score, Absatznummer
lines = []
for title, score, number in hits:
    lines.append(f"{score:.2f} {title}")
# Die Meldung lautete: ValueError: too many values to unpack (expected 2).
# Jedes Tupel liefert drei Werte, links standen nur zwei Namen.
print(lines)
```

[↑ zur Aufgabenstellung](#)

 Aufgabe 2.3 #2

5 min

Die Vorgabe in der Zelle darunter legt `record` an, eine Kopie des Absatzes 108. Arbeiten Sie nur mit `record`, die Kursdaten in `passages` bleiben unverändert.

1. Speichern Sie den Titel in `title` und die Anzahl der Zeichen des Textes in `n_chars`. Gemeint ist der Text des Absatzes unter dem Schlüssel `"text"`, nicht der Titel.
2. Ergänzen Sie `record` um den Schlüssel `"chars"` mit diesem Wert.
3. Speichern Sie in `has_author`, ob `record` den Schlüssel `"author"` besitzt.

Zusatzaufgabe: Geben Sie in der Zelle nach der Prüfwelle mit einer Schleife über `record.items()` zu jedem Schlüssel den Typ des Werts aus, zum Beispiel `title <class 'str'>`.

```
record = passages[108].copy()      # Vorgabe: eine eigene Kopie des Dicts, wie copy() bei einer Liste
title = record["title"]
n_chars = len(record["text"])
record["chars"] = n_chars
has_author = "author" in record
```

```
# Zusatzaufgabe: zu jedem Schlüssel von record den Typ des Werts ausgeben
for key, value in record.items():
    print(key, type(value))
```

[↑ zur Aufgabenstellung](#) Aufgabe 2.3 #3

5 min

Schreiben Sie die Funktion `index_by_id(records)`. Sie bekommt eine Liste von Absätzen und gibt ein Dict zurück, das jeder `id` den zugehörigen Absatz zuordnet, also das ganze Dict des Absatzes. Probieren Sie Ihre Funktion zuerst an `sample` aus und sehen Sie sich das Ergebnis an.

```
def index_by_id(records):
    result = {}
    for p in records:
        result[p["id"]] = p
    return result
```

[↑ zur Aufgabenstellung](#)

 Aufgabe 2.3 #4

10 min

Review. Ein Kollege möchte Absätze über den Titel nachschlagen und legt die Funktion `index_by_title` in der Zelle darunter vor. Die Anforderung lautet: Zu jedem Titel sollen **alle** Absätze des Artikels auffindbar sein.

1. Führen Sie die Zelle aus und wenden Sie die Funktion in einer neuen Zelle auf `sample` an. Vergleichen Sie die Anzahl der Einträge mit der Anzahl der Absätze.
2. Notieren Sie Ihr Urteil als Kommentar: Erfüllt die Funktion die Anforderung?
3. Schreiben Sie `group_by_title(records)`. Sie gibt ein Dict zurück, das jedem Titel die Liste der Nummern (`id`) aller Absätze mit diesem Titel zuordnet. Abgelegt werden nur die Nummern, nicht die ganzen Absätze wie bei `index_by_id`. Die Nummern stehen in der Reihenfolge, in der die Absätze in `records` vorkommen. Das Ergebnis ist aufgebaut wie `groups_demo` in der Zelle oben. Sehen Sie es sich für `sample` mit `pprint` an.

Zusatzaufgabe: Bestimmen Sie in der Zelle nach der Prü fzelle mit einer Schleife über `group_by_title(passages).items()`, welcher Titel die meisten Absätze hat und wie viele es sind.

```
# Die zu prüfende Fassung
def index_by_title(records):
    result = {}
    for p in records:
        result[p["title"]] = p
    return result

# Ihr Urteil als Kommentar:
# Die Funktion erfüllt die Anforderung nicht. Ein Dict hat je Schlüssel genau einen Wert.
# Kommt ein Titel zum zweiten Mal vor, ersetzt die Zuweisung den bisherigen Absatz, ohne
# Fehlermeldung. Für sample entstehen 2 Einträge aus 3 Absätzen, für alle Daten 217 aus 400.

def group_by_title(records):
    result = {}
    for p in records:
        title = p["title"]
        if title not in result:
            result[title] = []
        result[title].append(p["id"])
    return result
```

```
# Zusatzaufgabe: der Titel mit den meisten Absätzen
best_title, best_count = "", 0
for title, ids in group_by_title(passages).items():
    if len(ids) > best_count:
        best_title, best_count = title, len(ids)
print(best_title, best_count)
# Ausgabe: Mexiko-Stadt 6. Der Artikel Aufklärung hat ebenfalls 6 Absätze, die Schleife
# behält bei Gleichstand den ersten Titel.
```

[↑ zur Aufgabenstellung](#)

Lernfragen 2.3 #1

1. Was gibt `len(index_by_title(passages))` zurück, und warum ist es weniger als 400?

1. Ein Dict hat je Schlüssel genau einen Eintrag. Die 400 Absätze haben nur 217 verschiedene Titel. Jede Zuweisung an einen schon vorhandenen Titel ersetzt den bisherigen Wert.

2. Welche Aussagen über Dicts stimmen?

- ✓ `"title" in p` prüft, ob es den Schlüssel `"title"` gibt. — `in` prüft bei einem Dict die Schlüssel.
- ✗ `p["author"]` liefert `None`, wenn der Schlüssel fehlt. — Der Zugriff bricht mit einem `KeyError` ab. `None` liefert `p.get("author")`.
- ✓ Eine Zuweisung an einen vorhandenen Schlüssel ersetzt den Wert. — Je Schlüssel gibt es genau einen Eintrag.
- ✗ Ein Dict darf jeden Wert als Schlüssel verwenden. — Schlüssel müssen hashbar sein. Eine Liste ist es nicht.

[↑ zur Aufgabenstellung](#)

2.4 Text verarbeiten, zählen, Mengen bilden

Aufgabe 2.4 #1

2 min

Der Code in der Zelle darunter soll die Anfrage in Kleinbuchstaben umwandeln, das Fragezeichen entfernen und in Wörter zerlegen. Das erwartete Ergebnis ist `['wo', 'liegt', 'san', 'diego']`. Führen Sie die Zelle aus, vergleichen Sie und korrigieren Sie den Fehler direkt in der Zelle.

```
query = "Wo liegt San Diego?"
query = query.lower()
query = query.strip("?")
query_words = query.split()
# lower() und strip() geben neue Strings zurück. Ohne Zuweisung geht das Ergebnis verloren,
# und split() arbeitet auf dem unveränderten Text.
print(query_words)
```

[↑ zur Aufgabenstellung](#)

 Aufgabe 2.4 #2

10 min

Schreiben Sie die Funktion `clean_words(text)`. Sie gibt die Wörter eines Textes als Liste zurück und erfüllt drei Anforderungen:

1. Alle Wörter stehen in Kleinbuchstaben.
2. An Anfang und Ende jedes Worts sind die Zeichen aus `PUNCTUATION` entfernt.
3. Wörter, von denen danach nichts übrig bleibt, fehlen im Ergebnis.

Entwickeln Sie an kurzen Beispielen wie `"Die GPU, die gpu!"` und wenden Sie die Funktion danach auf `sample[1]["text"]` an.

Zusatzaufgabe: Geben Sie in der Zelle nach der Prü fzelle `clean_words(passages[108]["text"])` aus. Finden Sie die Wörter, an denen trotz der Bereinigung noch Zeichen hängen, die keine Buchstaben sind. Woran liegt das?

```
PUNCTUATION = ".,:;!()?\"'\"" # Vorgabe: Zeichen, die an den Wortenden entfernt werden

def clean_words(text):
    result = []
    for word in text.lower().split():
        word = word.strip(PUNCTUATION)
        if word != "":
            result.append(word)
    return result
```

```
# Zusatzaufgabe: An welchen Wörtern hängen noch Zeichen, die keine Buchstaben sind?
print(clean_words(passages[108]["text"]))
# In der Ausgabe stehen "'black", "mountain'", "'mount" und "soledad'". An ihnen hängen je
# zwei Apostrophe, ein Rest der Auszeichnung für Kursivschrift aus der Wikipedia. Der Apostroph
# steht nicht in PUNCTUATION, deshalb entfernt strip ihn nicht.
```

[↑ zur Aufgabenstellung](#)
 Kurz überlegen 2.4 #1

Die nächste Zelle sammelt die Wörter zweier Texte in einer Liste. Was gibt sie aus?

2. `append` hängt genau ein Element an, hier also jeweils die ganze Wortliste. Das Ergebnis ist eine Liste aus zwei Listen: `[[ 'die', 'gpu'], [ 'die', 'cpu', 'wartet'] ]`.

 Aufgabe 2.4 #3

2 min

Schreiben Sie die Funktion `count_words(words)`. Sie bekommt eine Liste von Wörtern und gibt ein Dict zurück, das jedem Wort die Anzahl seiner Vorkommen zuordnet. Die Zelle darunter wendet Ihre Funktion danach auf die Wörter aller 400 Absätze an.

```
def count_words(words):
    counts = {}
    for word in words:
        counts[word] = counts.get(word, 0) + 1
    return counts
```

[↑ zur Aufgabenstellung](#)

 Aufgabe 2.4 #4

10 min

Schreiben Sie die Funktion `shared_words(text_a, text_b)`. Sie gibt das Set der Wörter zurück, die in beiden Texten vorkommen. Verwenden Sie `clean_words`, damit Großschreibung und Satzzeichen keine Rolle spielen.

Bestimmen Sie danach das Vokabular des Korpus: Legen Sie die neue Variable `corpus_vocabulary` an, das Set aller verschiedenen Wörter in `corpus_words`. Die Liste `corpus_words` mit den Wörtern aller 400 Absätze stammt aus der Zelle nach der Aufgabe zu `count_words`.

Zusatzaufgabe: Die Anfrage 0 lautet „Wo liegt San Diego?“. Berechnen Sie in der Zelle nach der Prüfwelle `shared_words(queries[0][\"text\"], passages[108][\"text\"])` und `shared_words(queries[0][\"text\"], passages[6][\"text\"])`. Der erste Absatz ist der gesuchte, der zweite handelt von einem Museum in Mexiko-Stadt. Wie viele Wörter teilt die Anfrage mit jedem der beiden Absätze? Suchen Sie im Text des Absatzes 6 die Stellen, an denen die gemeinsamen Wörter vorkommen.

```
def shared_words(text_a, text_b):
    return set(clean_words(text_a)) & set(clean_words(text_b))
```

```
# das Set aller verschiedenen Wörter in corpus_words
corpus_vocabulary = set(corpus_words)
```

```
# Zusatzaufgabe: Wie viele Wörter teilt die Anfrage 0 mit den Absätzen 108 und 6?
print(shared_words(queries[0][\"text\"], passages[108][\"text\"]))
print(shared_words(queries[0][\"text\"], passages[6][\"text\"]))
# Mit dem gesuchten Absatz 108 teilt die Anfrage 'liegt', 'san' und 'diego', mit Absatz 6 teilt
# sie 'wo', 'san' und 'diego'. Beide Absätze kommen auf drei gemeinsame Wörter. In Absatz 6
# stammen 'diego' und 'san' aus „Diego Rivera“ und „San Angel“, mit der Stadt San Diego hat der
# Absatz nichts zu tun. Gezählte Wörter allein können die beiden Absätze nicht unterscheiden.
```

[↑ zur Aufgabenstellung](#)

Lernfragen 2.4 #2

1. Warum liefert die Stichwortsuche mit `clean_words` mehr richtige Treffer als mit `lower().split()` allein?

Ohne die Bereinigung bleiben Satzzeichen am Wort hängen. Aus „Diego?“ in der Anfrage wird `diego?`, im Absatz steht `diego`. Für das Set sind das zwei verschiedene Wörter, die Überschneidung wird also kleiner, als sie inhaltlich ist. Mit `clean_words` zählen beide als dasselbe Wort.

2. Was ist der Unterschied zwischen `{}` und `set()`?

`{}` ist ein leeres Dict, `set()` ist ein leeres Set. Geschweifte Klammern mit Werten, etwa `{\"gpu\", \"cpu\"}`, bilden ein Set. Nur die leeren Klammern sind für das Dict reserviert.

3. Sie haben eine Liste mit 100.000 Wörtern und wollen für viele einzelne Wörter prüfen, ob sie darin vorkommen. Welchen Container wählen Sie für die Prüfung, und warum?

Ein Set aus den Wörtern. Die Prüfung `word in s` hängt bei einem Set im Mittel nicht von der Größe ab, bei einer Liste wächst die Zeit mit der Länge. Das haben Sie im Termin gemessen.

[↑ zur Aufgabenstellung](#)

2.5 Slicing und Kopien

Kurz überlegen 2.5 #1

Ein Slice einer Liste ist eine neue Liste. Die nächste Zelle legt eine Auswahl aus drei Datensätzen an, entfernt mit `pop()` ein Element aus der Auswahl und ändert einen Titel. `pop()` steht im Listen-Werkzeugkasten von Teil 1: Die Methode entfernt das letzte Element einer Liste und gibt es zurück. Was gibt die Zelle aus?

3 ZEIT. Die Auswahl ist eine neue Liste, deshalb verliert `records` durch `pop` kein Element. Die neue Liste enthält aber dieselben Dicts wie `records`, keine Kopien. Die Titeländerung trifft deshalb den Datensatz, auf den beide Listen verweisen.

Aufgabe 2.5 #1

10 min

Schreiben Sie die Funktion `copy_first(records, n)`. Sie gibt die ersten `n` Datensätze als neue Liste zurück. Für `n` dürfen Sie eine ganze Zahl ab 0 voraussetzen. Sind weniger als `n` Datensätze vorhanden, gibt die Funktion alle vorhandenen zurück, mit `n = 0` ist das Ergebnis leer. Jeder Datensatz im Ergebnis ist eine eigene Kopie, sodass eine Änderung am Ergebnis die Eingabe nie verändert. Die Datensätze sind Dicts mit Zahlen und Strings als Werten.

Schreiben Sie in die zweite Zelle einen eigenen Test: Er ändert einen Titel im Ergebnis und zeigt mit `assert`, dass der Titel in `test_records` unverändert ist. Arbeiten Sie dafür mit den Beispieldaten `test_records` aus der Vorgabe, nicht mit den Kursdaten.

```
def copy_first(records, n):
    result = []
    for r in records[:n]:
        result.append(r.copy())      # je Datensatz eine eigene Kopie
    return result
```

```
# Vorgabe: Beispieldaten für Ihren Test
test_records = [{"id": 1, "title": "Zeit"},
                {"id": 2, "title": "Koran"},
                {"id": 3, "title": "Gehirn"}]
```

```
# Ihr Test: einen Titel im Ergebnis ändern, dann zeigen, dass test_records unverändert ist
_part = copy_first(test_records, 2)
_part[0]["title"] = "ZEIT"
assert test_records[0]["title"] == "Zeit"      # die Änderung am Ergebnis erreicht die Eingabe nicht
# Mit dem Slice records[:n] allein schlage dieser Test fehl: Beide Listen enthielten dasselbe Dict.
```

[↑ zur Aufgabenstellung](#)

Lernfragen 2.5 #2

1. Welche Aussagen über `xs = [1, 2, 3, 4, 5]` stimmen?

- ✓ `xs[:2] + xs[2:]` ergibt wieder `[1, 2, 3, 4, 5]`. – Das Ende des ersten Slice ist ausgeschlossen, der zweite beginnt dort. Kein Element fehlt oder kommt doppelt vor.
- ✗ `xs[1:3]` enthält drei Elemente. – Es sind `3 - 1`, also zwei Elemente: die an den Positionen 1 und 2.
- ✓ `xs[10:]` liefert eine leere Liste. – Ein Slice über das Ende hinaus bricht nicht ab.
- ✗ `ys = xs[:]` bewirkt, dass `ys.append(6)` auch `xs` verlängert. – `xs[:]` ist eine neue Liste. `append` auf `ys` lässt `xs` unberührt.

2. `sample` enthält drei Absätze aus `passages`. Was bewirkt `sample[0]["title"] = "Test"` in den Kursdaten, und warum?

Auch `passages[6]["title"]` ist danach "Test". `sample` ist eine eigene Liste, sie enthält aber dieselben Dicts wie `passages`. Die Zuweisung verändert das Dict, auf das beide Listen verweisen. Aus diesem Grund arbeiten die Experimente dieses Kapitels mit eigenen Beispieldaten.

[↑ zur Aufgabenstellung](#)

2.6 enumerate, zip und Comprehensions

Aufgabe 2.6 #1

10 min

Schreiben Sie beide Funktionen als List Comprehension.

- `titles_of(hits)` gibt die Titel einer Trefferliste zurück, in derselben Reihenfolge.
- `hits_from(hits, limit)` gibt die Treffer zurück, deren Score mindestens `limit` beträgt. Das Ergebnis enthält die vollständigen Paare.

Eine Trefferliste `hits` ist eine Liste von Paaren aus Titel und Score, zum Beispiel `[("Gehirn", 0.89), ("Zeit", 0.72)]`.

Zusatzaufgabe: Schreiben Sie `hits_from` in der Zelle nach der Prü fzelle zusätzlich als gewöhnliche Schleife unter dem Namen `hits_from_loop` und vergleichen Sie beide Fassungen mit `==` an einem Beispiel. Welche Fassung lesen Sie leichter?

```
def titles_of(hits):
    return [title for title, score in hits]

def hits_from(hits, limit):
    return [(title, score) for title, score in hits if score >= limit]
```

```
# Zusatzaufgabe: hits_from als gewöhnliche Schleife, danach beide Fassungen vergleichen
def hits_from_loop(hits, limit):
    result = []
    for title, score in hits:
        if score >= limit:
            result.append((title, score))
    return result

example = [("Gehirn", 0.89), ("Zeit", 0.72), ("Koran", 0.65)]
print(hits_from_loop(example, 0.7) == hits_from(example, 0.7))
# Beide Fassungen liefern dasselbe. Hier ist die Comprehension gut lesbar: ein Ausdruck,
# eine Schleife, eine Bedingung.
```

[↑ zur Aufgabenstellung](#)

Aufgabe 2.6 #2

10 min

Review. Die Funktion `long_words` in der Zelle darunter ist als verschachtelte Comprehension geschrieben. Sie verwendet Ihre Funktion `clean_words` aus dem Kapitel 2.4 und liefert zu einer Liste von Absätzen alle bereinigten Wörter mit mehr als zwölf Zeichen, die keinen Bindestrich enthalten.

1. Führen Sie die Zelle aus und wenden Sie die Funktion in einer neuen Zelle auf `sample` an.
2. Schreiben Sie `long_words_loop(records)` als gewöhnliche Schleife mit demselben Ergebnis.
3. Notieren Sie als Kommentar, welche Fassung Sie in einem Review schneller prüfen könnten, und warum.

```
# Die vorgelegte Fassung
def long_words(records):
    return [w for p in records for w in clean_words(p["text"]) if len(w) > 12 if "-" not in w]

def long_words_loop(records):
    result = []
    for p in records:
        for w in clean_words(p["text"]):
            if len(w) > 12 and "-" not in w:
                result.append(w)
    return result

# Welche Fassung könnten Sie schneller prüfen, und warum?
# Die Schleife. Die Reihenfolge der Schleifen und die beiden Bedingungen stehen je in einer
# eigenen Zeile, und zwischen den Zeilen lassen sich zum Prüfen Ausgaben einfügen. In der
# Comprehension muss man die Reihenfolge der for-Teile kennen und die Bedingungen im Kopf verbinden.
```

[↑ zur Aufgabenstellung](#)

Lernfragen 2.6 #1

1. Was gibt `[len(w) for w in ["gpu", "tensor"] if len(w) > 3]` zurück?

`[6]`. Die Bedingung lässt nur "tensor" durch, und der Ausdruck vorn liefert dessen Anzahl der Zeichen.

2. Zwei Listen `passages` und `vectors` gehören elementweise zusammen. Durch einen Fehler beim Laden hat `vectors` nur 399 Einträge. Was passiert bei `for p, v in zip(passages, vectors):`, und wie fällt der Fehler auf?

Die Schleife läuft ohne Fehlermeldung 399-mal, der letzte Absatz wird übergangen. Der Fehler fällt nur auf, wenn man die Anzahlen vergleicht oder `zip(passages, vectors, strict=True)` schreibt. Dann bricht `zip` mit einem `ValueError` ab.

[↑ zur Aufgabenstellung](#)

2.7 Sortieren und Funktionen als Argumente

Kurz überlegen 2.7 #1

Eine Trefferliste besteht aus Paaren `(title, score)`. Wonach ordnet `sorted(hits)`, wenn Sie nichts weiter angeben?

Nach dem Titel. Tupel werden Position für Position verglichen: zuerst die ersten Werte, und nur bei Gleichheit die zweiten. `sorted(hits)` ordnet die Treffer also alphabetisch nach dem Titel, der Score entscheidet nur zwischen Treffern mit gleichem Titel.

 Aufgabe 2.7 #1

10 min

Schreiben Sie die Funktion `top_k(hits, k)`. Sie gibt die `k` Treffer mit dem höchsten Score zurück, absteigend nach dem Score geordnet. Für `k` dürfen Sie eine ganze Zahl ab 0 voraussetzen. Sind weniger als `k` Treffer vorhanden, gibt die Funktion alle vorhandenen zurück, mit `k = 0` ist das Ergebnis leer. Bei gleichem Score bleibt die Reihenfolge der Eingabe erhalten. Die Eingabeliste bleibt unverändert. Im Termin hat diese Aufgabe eine Schleife mit Arbeitskopie gebraucht.

Zusatzaufgabe: Im Kapitel [2.3](#) hat die Prü fzelle zu `group_by_title` das Dict `groups` angelegt. Es ordnet jedem Titel die Liste seiner Absatznummern zu. Voraussetzung ist, dass Sie die Aufgabe dort gelöst haben. Geben Sie in der Zelle nach der Prü fzelle die fünf Titel mit den meisten Absätzen aus. Bei gleicher Anzahl soll der Titel alphabetisch entscheiden. Als Schlüssel eignet sich dafür ein Tupel aus der negativen Anzahl und dem Titel.

```
def top_k(hits, k):
    return sorted(hits, key=lambda hit: hit[1], reverse=True)[:k]

# Zusatzaufgabe: die fünf Titel mit den meisten Absätzen
ranked = sorted(groups.items(), key=lambda item: (-len(item[1]), item[0]))[:5]
for title, ids in ranked:
    print(len(ids), title)
# Ausgabe: 6 Aufklärung, 6 Mexiko-Stadt, 5 Bern, 5 Hannover, 5 Haushund.
# Der Schlüssel ist ein Tupel: Erst entscheidet die negative Anzahl (viele Absätze zuerst),
# bei Gleichstand der Titel.
```

[↑ zur Aufgabenstellung](#) Aufgabe 2.7 #2

10 min

Review. Die Fassung `top_k_v2` in der Zelle darunter soll dieselben Anforderungen erfüllen wie `top_k`: die `k` besten Treffer, absteigend geordnet, Gleichstände in der Reihenfolge der Eingabe, Eingabeliste unverändert.

1. Führen Sie die Zelle aus und probieren Sie die Funktion in einer neuen Zelle aus.
2. Schreiben Sie in die zweite Zelle einen eigenen Test, der zeigt, ob die Funktion alle Anforderungen erfüllt.
3. Notieren Sie Ihr Urteil als Kommentar und korrigieren Sie die Funktion, falls nötig, direkt in der Zelle.

```
# Die zu prüfende Fassung – korrigieren Sie nur, was wirklich falsch ist
def top_k_v2(hits, k):
    ordered = sorted(hits, key=lambda hit: hit[1], reverse=True)
    return ordered[:k]

# Urteil: Die vorgelegte Fassung liefert die richtigen Treffer, verletzt aber die Anforderung
# „Eingabeliste unverändert“. hits.sort(...) ordnet die Liste des Aufrufers um. Die Korrektur
# verwendet sorted, das eine neue Liste liefert.

# Ihr Test:
_mine = [("Bern", 0.61), ("Koran", 0.80), ("Zeit", 0.74), ("Gehirn", 0.80)]
_copy = list(_mine) # Kopie zum Vergleich nach den Aufrufen
_top = top_k_v2(_mine, 2)
assert _top == [("Koran", 0.80), ("Gehirn", 0.80)] # Normalfall und Rückweg: die zwei besten kommen
per return an
assert top_k_v2(_mine, 0) == [] # Grenzfall: k = 0
assert len(top_k_v2(_mine, 10)) == 4 # Grenzfall: k größer als die Liste
assert _mine == _copy # Nebenwirkungen: die Eingabe muss unverändert
sein
# Mit der vorgelegten Fassung bestehen die ersten drei Tests, nur der letzte schlägt fehl:
# _mine ist nach dem Aufruf nach dem Score geordnet. Ein Test, der nur das Ergebnis prüft,
# hätte den Fehler nicht gefunden.
```

[↑ zur Aufgabenstellung](#)

Lernfragen 2.7 #2

1. Welche Zeilen ordnen `words` nach der Anzahl der Zeichen und liefern eine Liste?

- ✓ `sorted(words, key=len)` – Übergeben wird die Funktion `len`. Das Ergebnis ist eine neue Liste.
- ✗ `sorted(words, key=len())` – `len()` wird sofort ohne Argument aufgerufen und bricht mit einem `TypeError` ab.
- ✗ `words.sort(key=len)` – Die Zeile ordnet `words` richtig, gibt aber `None` zurück und keine Liste.
- ✓ `sorted(words, key=lambda w: len(w))` – Das `lambda` leistet dasselbe wie `len`. Kürzer ist `key=len`.

2. `sorted(hits, key=lambda hit: hit[1], reverse=True)[: -1]` und `sorted(hits, key=lambda hit: hit[1])` ordnen beide aufsteigend nach dem Score. Liefern sie immer dasselbe Ergebnis?

Nein, nicht bei Gleichständen. `sorted` behält bei gleichem Schlüssel die Reihenfolge der Eingabe bei, auch mit `reverse=True`. Das nachträgliche Umdrehen mit `[ : -1]` kehrt auch die Reihenfolge der gleichrangigen Treffer um.

[↑ zur Aufgabenstellung](#)

2.8 Abschluss

Aufgabe 2.8 #1

15 min

Schreiben Sie die Funktion `select_hits(hits, threshold, k)`. Sie wählt aus einer Trefferliste aus und erfüllt vier Anforderungen:

1. Übernommen werden die Treffer, deren Score mindestens `threshold` beträgt.
2. Das Ergebnis ist absteigend nach dem Score geordnet. Bei gleichem Score bleibt die Reihenfolge der Eingabe erhalten.
3. Zurückgegeben werden höchstens `k` Treffer. `k` ist eine ganze Zahl ab 0, mit `k = 0` ist das Ergebnis eine leere Liste.
4. Die Eingabeliste bleibt unverändert.

Schreiben Sie in die zweite Zelle zu jeder Anforderung mindestens einen eigenen Test mit `assert` und notieren Sie als Kommentar dahinter, welche Anforderung der Test prüft. Führen Sie erst danach die Prüfselle aus.

```
def select_hits(hits, threshold, k):
    selected = [hit for hit in hits if hit[1] >= threshold]           # auswählen
    ordered = sorted(selected, key=lambda hit: hit[1], reverse=True) # ordnen
    return ordered[:k]                                              # kürzen

# Ihre Tests: je Anforderung mindestens ein assert, dahinter als Kommentar die geprüfte Anforderung
assert ("Iran", 0.50) in select_hits(demo_hits, 0.5, 10)           # 1: die Grenze selbst gehört dazu
assert ("Sichuan", 0.42) not in select_hits(demo_hits, 0.5, 10)    # 1: unter der Grenze fällt weg
assert select_hits(demo_hits, 0.5, 2) == [("Koran", 0.80), ("Gehirn", 0.80)] # 2: Gleichstand
assert select_hits(demo_hits, 0.5, 3)[2] == ("Zeit", 0.74)        # 2: absteigend geordnet
assert len(select_hits(demo_hits, 0.0, 4)) == 4                   # 3: höchstens k Treffer
assert select_hits(demo_hits, 0.5, 0) == []                       # 3: k = 0
_mine = [("Bern", 0.61), ("Koran", 0.80), ("Zeit", 0.74)]        # 4: eigene, ungeordnete Liste
_copy = list(_mine)                                               # Kopie vor dem ersten Aufruf
select_hits(_mine, 0.5, 3)
assert _mine == _copy                                             # 4: die Eingabe bleibt unverändert

# Für Anforderung 4 eignet sich demo_hits nicht mehr: Eine Funktion, die ihre Eingabe umordnet,
# hätte demo_hits schon beim ersten Aufruf oben verändert, und eine Kopie von hier wäre die
# bereits veränderte Liste. Deshalb eine frische Liste und die Kopie vor dem ersten Aufruf.
```

[↑ zur Aufgabenstellung](#)

 Aufgabe 2.8 #2

10 min

Review mit offenem Ausgang. Die Fassung `select_hits_v2` in der Zelle darunter soll dieselben vier Anforderungen erfüllen. Entscheiden Sie wie in einem echten Review:

1. Lesen Sie die Funktion und führen Sie die Zelle aus.
2. Prüfen Sie jede der vier Anforderungen mit einem eigenen Test in der zweiten Zelle. Ihre Tests aus der vorigen Aufgabe können Sie anpassen.
3. Notieren Sie Ihr Urteil als Kommentar und begründen Sie es am Code: Warum erfüllt die Funktion jede Anforderung, oder an welcher Stelle verletzt sie eine? Korrigieren Sie nur, was wirklich falsch ist.

Führen Sie die Zelle auf jeden Fall aus, auch wenn Sie nichts korrigieren.

```
# Ihre Tests und Ihr Urteil als Kommentar:
assert select_hits_v2(demo_hits, 0.5, 10) == select_hits(demo_hits, 0.5, 10)    # 1 und 2 im Normalfall
assert ("Iran", 0.50) in select_hits_v2(demo_hits, 0.5, 10)                  # 1: die Grenze gehört dazu
assert select_hits_v2(demo_hits, 0.9, 3) == []                               # 1: kein Treffer erreicht die Grenze
assert select_hits_v2(demo_hits, 0.5, 2) == [("Koran", 0.80), ("Gehirn", 0.80)] # 2: Gleichstand
assert select_hits_v2(demo_hits, 0.5, 0) == []                               # 3: k = 0
_mine = [("Bern", 0.61), ("Koran", 0.80), ("Zeit", 0.74)]                   # 4: eigene, ungeordnete Liste
_copy = list(_mine)                                                           # Kopie vor dem ersten Aufruf
select_hits_v2(_mine, 0.5, 3)
assert _mine == _copy                                                         # 4: Eingabe unverändert

# Urteil: Die Fassung erfüllt alle vier Anforderungen, es gibt nichts zu korrigieren.
# Sie sieht ungewohnt aus, ist aber korrekt:
# - sorted liefert eine neue Liste. Das pop() danach verändert nur diese neue Liste.
# - Der Schlüssel -hit[1] ordnet absteigend. Gleichstände behalten die Reihenfolge der Eingabe.
# - Nach dem Ordnen stehen die Treffer unter der Grenze am Ende. Die Schleife entfernt sie von hinten.
# - Ist die Liste leer, ist len(result) > 0 falsch. Python wertet den rechten Teil von and dann nicht
#   mehr aus, der Zugriff result[-1] findet nicht statt. So verhält sich auch && in C++.
```

[↑ zur Aufgabenstellung](#)
 Aufgabe 2.8 #3

freiwillig, 10 min

Zusatzaufgabe zu Klassen. Im Termin haben Sie die Klasse `Corpus` um Methoden ergänzt. Diese Aufgabe wiederholt die Schreibweise an einem kleineren Beispiel. Schreiben Sie die Klasse `HitList`:

1. `__init__(self, hits)` legt die Trefferliste im Attribut `hits` ab.
2. Die Methode `select(self, threshold, k)` gibt die Auswahl nach den vier Anforderungen zurück. Sie darf `select_hits` aufrufen.
3. Die Sondermethode `__len__(self)` gibt die Anzahl der Treffer zurück.

```
class HitList:
    def __init__(self, hits):
        self.hits = hits

    def select(self, threshold, k):
        return select_hits(self.hits, threshold, k)

    def __len__(self):
        return len(self.hits)
```

[↑ zur Aufgabenstellung](#)